

Documentação completa com código-fonte

Este documento contém a explicação do funcionamento do projeto e o código-fonte integral de cada arquivo relevante, para apoio a apresentações e análise técnica.

Conteúdo

- Visão geral da arquitetura e fluxo de dados
- Código completo do frontend React/TypeScript
- Código completo do backend Express/SQLite
- Configuração de deploy (Vercel, Docker, Android)
- Índice de todos os arquivos incluídos

1. Visão geral do projeto

SaudeConnect é uma plataforma full-stack de saúde digital. O frontend React consome uma API REST Express protegida por JWT. O SQLite armazena usuários, unidades, agendamentos, prontuários, triagem, fila e auditoria. A mesma base pode rodar localmente, em Docker, na Vercel (serverless) ou como APK Android via Capacitor.

Arquitetura em camadas

Cliente (React + Vite) -> API REST (/api) -> Express (routes, auth, db) -> SQLite. Autenticação via JWT Bearer. Rotas /admin exigem role admin. Validação Zod no servidor.

Fluxo típico de uma requisicao

1. Usuário interage com formulário ou botão na interface React.
2. api.ts envia fetch com Authorization Bearer para /api/...
3. authRequired valida JWT e carrega usuário do banco.
4. routes.js valida body com Zod e executa operação SQL.
5. Resposta JSON atualiza estado React e re-renderiza a tela.

Índice de arquivos incluídos

- package.json (71 linhas)
- index.html (20 linhas)
- src/main.tsx (11 linhas)
- src/types.ts (196 linhas)
- src/api.ts (60 linhas)
- src/auth.tsx (133 linhas)
- src/App.tsx (1466 linhas)
- src/index.css (1639 linhas)
- server/db.js (539 linhas)
- server/auth.js (118 linhas)
- server/routes.js (839 linhas)
- server/index.js (115 linhas)
- api/[...path].js (4 linhas)
- vite.config.ts (13 linhas)
- capacitor.config.ts (13 linhas)
- vercel.json (24 linhas)
- Dockerfile (18 linhas)
- android/app/src/main/java/com/saudeconnect/app/MainActivity.java (6 linhas)
- android/app/src/main/AndroidManifest.xml (42 linhas)
- .dockerignore (11 linhas)
- .env.example (17 linhas)
- .gitattributes (7 linhas)
- .github/workflows/deploy-huggingface.yml (60 linhas)
- .gitignore (34 linhas)
- .vercel/project.json (1 linhas)
- android/.gitignore (102 linhas)
- android/app/.gitignore (3 linhas)
- android/app/build.gradle (55 linhas)
- android/app/capacitor.build.gradle (20 linhas)
- android/app/proguard-rules.pro (22 linhas)
- android/app/src/androidTest/java/com/getcapacitor/myapp/ExampleInstrumentedTest.java (27 linhas)
- android/app/src/main/assets/capacitor.config.json (9 linhas)
- android/app/src/main/assets/capacitor.plugins.json (2 linhas)
- android/app/src/main/assets/public/assets/index-BJfqpSh.js (11 linhas)
- android/app/src/main/assets/public/assets/index-PwuHWK31.css (2 linhas)
- android/app/src/main/assets/public/cordova.js (1 linhas)
- android/app/src/main/assets/public/cordova_plugins.js (1 linhas)
- android/app/src/main/assets/public/favicon.svg (5 linhas)
- android/app/src/main/assets/public/icons.svg (25 linhas)
- android/app/src/main/assets/public/index.html (21 linhas)

- android/app/src/main/assets/public/manifest.webmanifest (18 linhas)
- android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml (35 linhas)
- android/app/src/main/res/drawable/ic_launcher_background.xml (171 linhas)
- android/app/src/main/res/layout/activity_main.xml (13 linhas)
- android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml (5 linhas)
- android/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml (5 linhas)
- android/app/src/main/res/values/ic_launcher_background.xml (4 linhas)
- android/app/src/main/res/values/strings.xml (8 linhas)
- android/app/src/main/res/values/styles.xml (22 linhas)
- android/app/src/main/res/xml/config.xml (6 linhas)
- android/app/src/main/res/xml/file_paths.xml (5 linhas)
- android/app/src/test/java/com/getcapacitor/myapp/ExampleUnitTest.java (19 linhas)
- android/build.gradle (30 linhas)
- android/capacitor-cordova-android-plugins/build.gradle (59 linhas)
- android/capacitor-cordova-android-plugins/cordova.variables.gradle (7 linhas)
- android/capacitor-cordova-android-plugins/src/main/AndroidManifest.xml (8 linhas)
- android/capacitor.settings.gradle (4 linhas)
- android/gradle.properties (23 linhas)
- android/gradlew.bat (95 linhas)
- android/local.properties (2 linhas)
- android/settings.gradle (5 linhas)
- android/variables.gradle (16 linhas)
- artifacts/screenshots/debug.html (16 linhas)
- eslint.config.js (27 linhas)
- public/favicon.svg (5 linhas)
- public/icons.svg (25 linhas)
- public/manifest.webmanifest (18 linhas)
- scripts/generate-marketing-screenshots.mjs (199 linhas)
- scripts/smoke-visual.mjs (90 linhas)
- src/assets/react.svg (1 linhas)
- src/assets/vite.svg (2 linhas)
- tsconfig.app.json (26 linhas)
- tsconfig.json (8 linhas)
- tsconfig.node.json (25 linhas)

2. Configuração e entrada

Arquivos de configuração do projeto, ponto HTML e dependências npm.

Arquivo: package.json

Define nome, scripts npm, dependências e versoes do projeto. Scripts principais: dev (web+api), build, start, mobile:apk.

Total: 71 linhas | Linguagem: .json

```
{
  "name": "saudeconnectl",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "concurrently -n web,api -c cyan,green \"npm run dev:web\" \"npm run dev:api\"",
    "dev:web": "vite",
    "dev:api": "nodemon server/index.js",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "start": "node server/index.js",
    "preview": "npm run build && node server/index.js",
    "screenshots": "node scripts/generate-marketing-screenshots.mjs",
    "mobile:sync": "npm run build && npx cap sync android",
    "mobile:add": "npm run build && npx cap add android",
    "mobile:apk": "npm run mobile:sync && cd android && gradlew assembleDebug"
  },
  "dependencies": {
    "@capacitor/android": "^8.4.0",
    "@capacitor/cli": "^8.4.0",
    "@capacitor/core": "^8.4.0",
    "bcryptjs": "^3.0.3",
    "better-sqlite3": "^12.11.1",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "express-rate-limit": "^8.5.2",
    "helmet": "^8.2.0",
    "jsonwebtoken": "^9.0.3",
    "leaflet": "^1.9.4",
    "lucide-react": "^1.20.0",
    "morgan": "^1.11.0",
    "passport": "^0.7.0",
    "passport-google-oauth20": "^2.0.0",
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "react-leaflet": "^5.0.0",
    "react-router-dom": "^7.18.0",
    "zod": "^4.4.3"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/bcryptjs": "^2.4.6",
    "@types/cookie-parser": "^1.4.10",
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/jsonwebtoken": "^9.0.10",
    "@types/leaflet": "^1.9.21",
    "@types/morgan": "^1.9.10",
    "@types/node": "^24.12.3",
    "@types/passport": "^1.0.17",
    "@types/passport-google-oauth20": "^2.0.17",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.2",
    "concurrently": "^10.0.3",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "nodemon": "^3.1.14",
    "puppeteer-core": "^25.1.0",
    "typescript": "~6.0.2",
    "typescript-eslint": "^8.59.2",
    "vite": "^8.0.16",
    "wait-on": "^9.0.10"
  }
}
```

Arquivo: index.html

Ponto de entrada HTML da aplicação React. Carrega favicon, manifest PWA e monta o componente raiz em #root.

Total: 20 linhas | Linguagem: .html

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <link rel="manifest" href="/manifest.webmanifest" />
    <meta name="theme-color" content="#0f8b8d" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta
      name="description"
      content="SaudeConnect: portal integrado para pacientes, unidades de saude e administradores."
    />
    <title>SaudeConnect</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Arquivo: vite.config.ts

Configuração Vite com plugin React e proxy /api para localhost:3001 em dev.

Total: 13 linhas | Linguagem: .ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:3001',
    },
  },
})
```


Arquivo: capacitor.config.ts

Configuração Capacitor: appId, nome, webDir dist e esquema Android https.

Total: 13 linhas | Linguagem: .ts

```
import type { CapacitorConfig } from '@capacitor/cli';

const config: CapacitorConfig = {
  appId: 'com.saudeconnect.app',
  appName: 'SaudeConnect',
  webDir: 'dist',
  server: {
    androidScheme: 'https',
  },
};

export default config;
```

Arquivo: tsconfig.json

Configuração TypeScript raiz do monorepo.

Total: 8 linhas | Linguagem: json

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

Arquivo: tsconfig.app.json

Config TypeScript especifica da aplicação frontend.

Total: 26 linhas | Linguagem: .json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023", "DOM"],
    "module": "esnext",
    "types": ["vite/client"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}
```

Arquivo: tsconfig.node.json

Config TypeScript para scripts Node (Vite).

Total: 25 linhas | Linguagem: .json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": ".node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023"],
    "module": "esnext",
    "types": ["node"],
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["vite.config.ts"]
}
```

Arquivo: eslint.config.js

Regras ESLint para TypeScript e React.

Total: 27 linhas | Linguagem: .js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist', 'android', 'artifacts', 'data', 'node_modules']),
  {
    files: ['**/*.ts', '**/*.tsx'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
    },
    rules: {
      'react-hooks/set-state-in-effect': 'off',
      'react-refresh/only-export-components': 'off',
    },
  },
])
```

Arquivo: .env.example

Modelo de variáveis de ambiente (JWT, OAuth, DATA_DIR).

Total: 17 linhas | Linguagem: .example

```
PORT=3001
JWT_SECRET=troque-este-segredo
JWT_TTL=7d
DATA_DIR=./data
CLIENT_ORIGIN=http://localhost:5173, http://localhost:3001
CLIENT_URL=http://localhost:5173
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_CALLBACK_URL=http://localhost:3001/api/auth/google/callback
VITE_API_URL=

# Hugging Face Spaces example:
# PORT=7860
# CLIENT_URL=https://SEU-SPACE.hf.space
# CLIENT_ORIGIN=https://SEU-SPACE.hf.space
# GOOGLE_CALLBACK_URL=https://SEU-SPACE.hf.space/api/auth/google/callback
```

3. Frontend React (src/)

Interface do usuário: tipos, API client, autenticação, componentes e estilos.

Arquivo: src/main.tsx

Inicializa o React com StrictMode, importa estilos globais e renderiza App.

Total: 11 linhas | Linguagem: .tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Arquivo: src/types.ts

Tipos TypeScript compartilhados: User, Unit, Appointment, TriageCase, QueueEntry, DashboardPayload, AdminPayload e demais entidades da API.

Total: 196 linhas | Linguagem: .ts

```
export type Role = 'user' | 'admin';

export type User = {
  id: string;
  name: string;
  email: string;
  role: Role;
  avatar: string;
  provider: string;
  createdAt?: string;
  created_at?: string;
  lastLogin?: string;
  last_login?: string;
};

export type Unit = {
  id: string;
  name: string;
  type: string;
  city: string;
  district: string;
  address: string;
  phone: string;
  status: string;
  distance_km: number;
  hours: string;
  lat: number;
  lng: number;
  services: string[];
};

export type AppointmentStatus = 'pending' | 'confirmed' | 'completed' | 'cancelled';

export type Appointment = {
  id: string;
  user_id: string;
  unit_id: string;
  specialty: string;
  professional: string;
  scheduled_at: string;
  status: AppointmentStatus;
  reason: string;
  notes: string;
  created_at: string;
  unit_name?: string;
  unit_district?: string;
  unit_phone?: string;
  user_name?: string;
  user_email?: string;
};

export type Exam = {
  id: string;
  title: string;
  unit: string;
  requested_at: string;
  status: string;
  result_url: string | null;
};

export type RecordItem = {
  id: string;
  category: string;
  title: string;
  description: string;
  created_at: string;
};

export type PatientProfile = {
  user_id: string;
  cpf: string;
  birth_date: string;
  phone: string;
  sus_card: string;
  address: string;
  emergency_contact: string;
  updated_at: string;
};

export type TriageRisk = 'low' | 'medium' | 'high' | 'critical';
export type TriageStatus = 'waiting' | 'in_service' | 'resolved';

export type TriageCase = {
```



```

    id: string;
    user_id: string;
    symptoms: string;
    risk_level: TriageRisk;
    status: TriageStatus;
    recommendation: string;
    created_at: string;
    user_name?: string;
    user_email?: string;
  };

export type QueueStatus = 'waiting' | 'called' | 'done' | 'cancelled';

export type QueueEntry = {
  id: string;
  user_id: string;
  unit_id: string;
  service: string;
  position: number;
  estimated_minutes: number;
  status: QueueStatus;
  created_at: string;
  unit_name?: string;
  unit_address?: string;
  user_name?: string;
  user_email?: string;
};

export type TicketStatus = 'open' | 'in_review' | 'resolved';
export type Priority = 'low' | 'medium' | 'high';

export type Ticket = {
  id: string;
  user_id: string;
  subject: string;
  message: string;
  status: TicketStatus;
  priority: Priority;
  created_at: string;
  user_name?: string;
  user_email?: string;
};

export type Announcement = {
  id: string;
  title: string;
  body: string;
  audience: 'all' | 'users' | 'admins';
  published_at: string;
};

export type Integration = {
  id: string;
  name: string;
  status: 'online' | 'degraded' | 'offline';
  last_sync: string;
  latency_ms: number;
};

export type AuditLog = {
  id: string;
  actor_id: string | null;
  actor_name?: string;
  action: string;
  entity: string;
  entity_id: string | null;
  created_at: string;
};

export type DashboardPayload = {
  user: User;
  metrics: {
    nextAppointments: number;
    availableResults: number;
    activeTickets: number;
    connectedUnits: number;
    activeTriage: number;
    queuePosition: number;
  };
  appointments: Appointment[];
  exams: Exam[];
  records: RecordItem[];
  profile: PatientProfile;
  triage: TriageCase[];
  queue: QueueEntry[];
  tickets: Ticket[];
  units: Unit[];
  announcements: Announcement[];
};

export type AdminPayload = {

```

```
overview: {
  users: number;
  appointments: number;
  openTickets: number;
  integrationsOnline: number;
  triageWaiting: number;
  queueWaiting: number;
};
appointments: Appointment[];
users: User[];
tickets: Ticket[];
triage: TriageCase[];
queue: QueueEntry[];
integrations: Integration[];
announcements: Announcement[];
auditLogs: AuditLog[];
};

export type BootstrapPayload = {
  googleEnabled: boolean;
};
```

Arquivo: src/api.ts

Cliente HTTP centralizado. Adiciona JWT no header Authorization, serializa JSON, trata erros e expoe helpers googleAuthUrl e mediaUrl.

Total: 60 linhas | Linguagem: .ts

```
import type { BootstrapPayload, User } from './types';

type ApiOptions = Omit<RequestInit, 'body'> & {
  body?: unknown;
};

const API_BASE = import.meta.env.VITE_API_URL || '/api';
let tokenGetter: (() => string | null) | null = null;

export function setAuthTokenGetter(getter: () => string | null) {
  tokenGetter = getter;
}

export async function api<T>(path: string, options: ApiOptions = {}): Promise<T> {
  const headers = new Headers(options.headers);
  const token = tokenGetter?.();

  if (token) headers.set('Authorization', `Bearer ${token}`);

  let body: BodyInit | undefined;
  if (options.body !== undefined) {
    headers.set('Content-Type', 'application/json');
    body = JSON.stringify(options.body);
  }

  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers,
    body,
  });

  const payload = await response.json().catch(() => ({}));

  if (!response.ok) {
    throw new Error(payload.message || 'Nao foi possivel concluir a acao.');
```

```
  }
```

```
  return payload as T;
}
```

```
export function googleAuthUrl() {
  return `${API_BASE}/auth/google`;
}
```

```
export function mediaUrl(path: string) {
  if (!path || /^https?:\/\//.test(path)) return path;
  if (!path.startsWith('/')) return path;
  const origin = API_BASE.endsWith('/api') ? API_BASE.slice(0, -4) : '';
  return `${origin}${path}`;
}
```

```
export function fetchBootstrap() {
  return api<BootstrapPayload>('/bootstrap');
}
```

```
export type AuthResponse = {
  user: User;
  token: string;
};
```

Arquivo: src/auth.tsx

Contexto de autenticação React. Gerencia token no localStorage, login/registro/logout, refresh via /auth/me e AuthProvider/useAuth.

Total: 133 linhas | Linguagem: tsx

```
import { createContext, useCallback, useContext, useEffect, useMemo, useState } from 'react';
import { api, type AuthResponse, setAuthTokenGetter } from './api';
import type { User } from './types';

type Credentials = {
  email: string;
  password: string;
};

type RegisterInput = Credentials & {
  name: string;
};

type AuthContextValue = {
  user: User | null;
  token: string | null;
  loading: boolean;
  login: (credentials: Credentials) => Promise<User>;
  register: (input: RegisterInput) => Promise<User>;
  consumeToken: (token: string) => Promise<User>;
  logout: () => void;
  refresh: () => Promise<User | null>;
};

const storageKey = 'saudeconnect.token';
const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [token, setToken] = useState<string | null>(() => localStorage.getItem(storageKey));
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setAuthTokenGetter(() => localStorage.getItem(storageKey));
  }, []);

  const persistSession = useCallback((nextToken: string, nextUser: User) => {
    localStorage.setItem(storageKey, nextToken);
    setToken(nextToken);
    setUser(nextUser);
    return nextUser;
  }, []);

  const clearLocalSession = useCallback(() => {
    localStorage.removeItem(storageKey);
    setToken(null);
    setUser(null);
  }, []);

  const logout = useCallback(() => {
    if (localStorage.getItem(storageKey)) {
      void api('/auth/logout', { method: 'POST' }).catch(() => undefined);
    }
    clearLocalSession();
  }, [clearLocalSession]);

  const refresh = useCallback(async () => {
    const currentToken = localStorage.getItem(storageKey);
    if (!currentToken) {
      setLoading(false);
      return null;
    }

    try {
      const payload = await api<{ user: User }>('/auth/me');
      setToken(currentToken);
      setUser(payload.user);
      return payload.user;
    } catch {
      if (localStorage.getItem(storageKey) === currentToken) {
        clearLocalSession();
      }
      return null;
    } finally {
      setLoading(false);
    }
  }, [clearLocalSession]);

  useEffect(() => {
    if (window.location.pathname === '/auth/callback') {
      setLoading(false);
      return;
    }
  })
}
```

```

    void refresh();
  }, [refresh]);

  const login = useCallback(
    async (credentials: Credentials) => {
      const payload = await api<AuthResponse>('/auth/login', {
        method: 'POST',
        body: credentials,
      });
      return persistSession(payload.token, payload.user);
    },
    [persistSession],
  );

  const register = useCallback(
    async (input: RegisterInput) => {
      const payload = await api<AuthResponse>('/auth/register', {
        method: 'POST',
        body: input,
      });
      return persistSession(payload.token, payload.user);
    },
    [persistSession],
  );

  const consumeToken = useCallback(
    async (nextToken: string) => {
      localStorage.setItem(storageKey, nextToken);
      setToken(nextToken);
      const payload = await api<{ user: User }>('/auth/me');
      return persistSession(nextToken, payload.user);
    },
    [persistSession],
  );

  const value = useMemo(
    () => ({ user, token, loading, login, register, consumeToken, logout, refresh }),
    [user, token, loading, login, register, consumeToken, logout, refresh],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth deve ser usado dentro de AuthProvider.');
```

```

  return context;
}
```

Arquivo: src/App.tsx

Componente principal com rotas React Router, telas de login, portal do paciente (início, mapa, cadastro, prontuários, triagem, fila), painel administrativo e integração Leaflet para mapa de unidades.

Total: 1466 linhas | Linguagem: .tsx

```
// Linhas 1-314
import 'leaflet/dist/leaflet.css';
import L from 'leaflet';
import {
  Activity,
  AlertTriangle,
  Bell,
  CalendarDays,
  ChevronRight,
  CircleUserRound,
  ClipboardList,
  Download,
  FileText,
  HeartPulse,
  Hospital,
  KeyRound,
  LayoutDashboard,
  ListChecks,
  Megaphone,
  LockKeyhole,
  Logout,
  Mail,
  Map,
  Navigation,
  Plus,
  RefreshCw,
  Search,
  Save,
  ShieldCheck,
  Stethoscope,
  UserCog,
  UserPlus,
  UsersRound,
  Wifi,
  Wrench,
} from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import { MapContainer, Marker, Popup, TileLayer, ZoomControl } from 'react-leaflet';
import { BrowserRouter, Navigate, NavLink, Route, Routes, useNavigate } from 'react-router-dom';
import { api, fetchBootstrap, googleAuthUrl, mediaUrl } from './api';
import { AuthProvider, useAuth } from './auth';
import type {
  AdminPayload,
  Appointment,
  AppointmentStatus,
  BootstrapPayload,
  DashboardPayload,
  Integration,
  PatientProfile,
  QueueEntry,
  QueueStatus,
  RecordItem,
  TriageCase,
  TriageRisk,
  TriageStatus,
  TicketStatus,
  Unit,
  User,
} from './types';

const markerIcon = L.divIcon({
  className: 'health-marker',
  html: '<span></span>',
  iconSize: [34, 34],
  iconAnchor: [17, 17],
});

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<HomeRedirect />} />
          <Route path="/login" element={<LoginPage />} />
          <Route path="/auth/callback" element={<AuthCallback />} />
          <Route
            path="/app"
            element={
              <ProtectedRoute>
                <PortalPage page="home" />
              </ProtectedRoute>
            }
          />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}
```

```

    <Route
      path="/app/mapa"
      element={
        <ProtectedRoute>
          <PortalPage page="map" />
        </ProtectedRoute>
      }
    />
    <Route
      path="/app/cadastro"
      element={
        <ProtectedRoute>
          <PortalPage page="profile" />
        </ProtectedRoute>
      }
    />
    <Route
      path="/app/prontuarios"
      element={
        <ProtectedRoute>
          <PortalPage page="records" />
        </ProtectedRoute>
      }
    />
    <Route
      path="/app/triagem"
      element={
        <ProtectedRoute>
          <PortalPage page="triage" />
        </ProtectedRoute>
      }
    />
    <Route
      path="/app/fila"
      element={
        <ProtectedRoute>
          <PortalPage page="queue" />
        </ProtectedRoute>
      }
    />
    <Route
      path="/admin"
      element={
        <ProtectedRoute role="admin">
          <AdminDashboard />
        </ProtectedRoute>
      }
    />
    <Route path="" element={<Navigate to="/" replace />} />
  </Routes>
</AuthProvider>
</BrowserRouter>
);
}

function HomeRedirect() {
  const { user, loading } = useAuth();
  if (loading) return <FullPageLoader />;
  if (!user) return <Navigate to="/login" replace />;
  return <Navigate to={user.role === 'admin' ? '/admin' : '/app'} replace />;
}

function ProtectedRoute({ children, role }: { children: React.ReactNode; role?: 'admin' }) {
  const { user, loading } = useAuth();
  if (loading) return <FullPageLoader />;
  if (!user) return <Navigate to="/login" replace />;
  if (role && user.role !== role) return <Navigate to="/app" replace />;
  return children;
}

function AuthCallback() {
  const navigate = useNavigate();

  useEffect(() => {
    const token = new URLSearchParams(window.location.search).get('token');
    const isNewUser = new URLSearchParams(window.location.search).get('newUser') === '1';
    if (!token) {
      navigate('/login', { replace: true });
      return;
    }

    localStorage.setItem('saudeconnect.token', token);
    window.location.replace(isNewUser ? '/app/triagem' : '/app');
  }, [navigate]);

  return <FullPageLoader label="Conectando sua conta..." />;
}

function LoginPage() {
  const { login, register } = useAuth();
  const navigate = useNavigate();

```

```

const [mode, setMode] = useState<'login' | 'register'>('login');
const [bootstrap, setBootstrap] = useState<BootstrapPayload | null>(null);
const [name, setName] = useState('');
const [email, setEmail] = useState('');
const [password, setPassword] = useState('');
const [message, setMessage] = useState('');
const [busy, setBusy] = useState(false);

useEffect(() => {
  fetchBootstrap().then(setBootstrap).catch(() => undefined);
}, []);

async function submit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  setBusy(true);
  setMessage('');

  try {
    const user =
      mode === 'login'
        ? await login({ email, password })
        : await register({ name, email, password });
    navigate(user.role === 'admin' ? '/admin' : mode === 'register' ? '/app/triagem' : '/app', { replace: true });
  } catch (error) {
    setMessage(error instanceof Error ? error.message : 'Falha ao autenticar.');
```

```

  } finally {
    setBusy(false);
  }
}

return (
  <main className="auth-shell">
    <section className="auth-visual" aria-label="Resumo da plataforma">
      <BrandLockup />
      <div className="live-board">
        <div className="live-board-header">
          <span>Rede conectada</span>
          <strong>94%</strong>
        </div>
        <div className="pulse-line">
          <i />
          <i />
          <i />
          <i />
        </div>
        <div className="mini-grid">
          <div>
            <Hospital size={20} />
            <strong>32</strong>
            <span>unidades</span>
          </div>
          <div>
            <CalendarDays size={20} />
            <strong>148</strong>
            <span>agendamentos</span>
          </div>
          <div>
            <FileText size={20} />
            <strong>2.4k</strong>
            <span>registros</span>
          </div>
        </div>
      </div>
      <div className="auth-copy">
        <h1>SaúdeConnect</h1>
        <p>
          Sistema integrado para atendimento, mapa de unidades, cadastro, prontuários,
          triagem digital e acompanhamento da fila.
        </p>
      </div>
    </section>

    <section className="auth-panel">
      <div className="auth-card">
        <div className="segmented" role="tablist" aria-label="Tipo de acesso">
          <button className={mode === 'login' ? 'active' : ''} onClick={() => setMode('login')} type="button">
            <LockKeyhole size={16} />
            Entrar
          </button>
          <button
            className={mode === 'register' ? 'active' : ''}
            onClick={() => setMode('register')}
            type="button">
            >
            <UserPlus size={16} />
            Criar conta
          </button>
        </div>

        <form onSubmit={submit} className="auth-form">
          <div>
```



```

    <p className="eyebrow">Acesso seguro</p>
    <h2>{mode === 'login' ? 'Entre no portal' : 'Crie seu cadastro'}</h2>
  </div>

  {mode === 'register' && (
    <label>
      Nome completo
      <span>
        <CircleUserRound size={18} />
        <input value={name} onChange={(event) => setName(event.target.value)} required />
      </span>
    </label>
  )}

  <label>
    E-mail
    <span>
      <Mail size={18} />
      <input
        value={email}
        onChange={(event) => setEmail(event.target.value)}
        type="email"
        autoComplete="email"
        required
      />
    </span>
  </label>

  <label>
    Senha
    <span>
      <KeyRound size={18} />
      <input
        value={password}
        onChange={(event) => setPassword(event.target.value)}
        type="password"
        autoComplete={mode === 'login' ? 'current-password' : 'new-password'}
        required
      />
    </span>
  </label>
  {mode === 'register' && (
    <small className="field-hint">Use pelo menos 10 caracteres com letra maiuscula, minuscula, numero e sim...
  )}

  {message && <div className="form-message error">{message}</div>}

  <button className="primary-action" type="submit" disabled={busy}>
    {busy ? <RefreshCw size={18} className="spin" /> : <ChevronRight size={18} />}
  </button>

// Linhas 315-562
  {mode === 'login' ? 'Entrar agora' : 'Cadastrar e entrar'}
</button>
</form>

<div className="oauth-area">
  <a className="google-button" href={googleAuthUrl()}>
    <ShieldCheck size={18} />
    Entrar com Google
  </a>
  {!bootstrap?.googleEnabled && (
    <small>O fluxo está pronto; falta cadastrar as credenciais OAuth no ambiente publicado.</small>
  )}
</div>

</div>
</section>
</main>
);
}

type PortalPageName = 'home' | 'map' | 'profile' | 'records' | 'triage' | 'queue';

function PortalPage({ page }: { page: PortalPageName }) {
  const [data, setData] = useState<DashboardPayload | null>(null);
  const [error, setError] = useState('');

  const load = () =>
    api<DashboardPayload>('/dashboard')
      .then(setData)
      .catch((err) => setError(err instanceof Error ? err.message : 'Erro ao carregar dados.'));

  useEffect(() => {
    void load();
  }, []);

  if (!data) {
    return <DashboardShell title="SaúdeConnect">{error ? <EmptyState message={error} /> : <PanelLoader />}</Dashboard...
  }

  return (

```

```

<DashboardShell title={pageTitle(page)} subtitle={pageSubtitle(page)}>
  {page === 'home' && <HomePage data={data} onReload={load} />}
  {page === 'map' && <MapPage units={data.units} />}
  {page === 'profile' && <ProfilePage profile={data.profile} onSaved={load} />}
  {page === 'records' && <RecordsPage records={data.records} exams={data.exams} onCreated={load} />}
  {page === 'triage' && <TriagePage triage={data.triage} onCreated={load} />}
  {page === 'queue' && <QueuePage queue={data.queue} units={data.units} onCreated={load} />}
</DashboardShell>
);
}

function HomePage({ data, onReload }: { data: DashboardPayload; onReload: () => Promise<void> }) {
  return (
    <>
      <section className="metrics-grid">
        <MetricCard icon={<CalendarDays />} label="Próximas consultas" value={data.metrics.nextAppointments} tone="bl...
        <MetricCard icon={<FileText />} label="Resultados disponiveis" value={data.metrics.availableResults} tone="gr...
        <MetricCard icon={<Stethoscope />} label="Triagens ativas" value={data.metrics.activeTriage} tone="orange" />
        <MetricCard icon={<ListChecks />} label="Posição na fila" value={data.metrics.queuePosition} tone="purple" />
      </section>

      <section className="content-grid">
        <div className="stack">
          <SectionHeader icon={<CalendarDays />} title="Sua agenda" />
          <div className="card-list">
            {data.appointments.map((appointment) => (
              <AppointmentCard key={appointment.id} appointment={appointment} />
            ))}
          </div>
        </div>
        <AppointmentForm units={data.units} onCreated={onReload} />
      </section>

      <section className="units-band">
        <SectionHeader icon={<Bell />} title="Comunicados" />
        <div className="unit-grid">
          {data.announcements.map((announcement) => (
            <article className="unit-card" key={announcement.id}>
              <div className="unit-topline">
                <strong>{announcement.title}</strong>
                <span>{formatDate(announcement.published_at, false)}</span>
              </div>
              <p>{announcement.body}</p>
            </article>
          ))}
        </div>
      </section>
    </>
  );
}

function MapPage({ units }: { units: Unit[] }) {
  const [selectedId, setSelectedId] = useState(units[0]?.id || '');
  const selected = units.find((unit) => unit.id === selectedId) || units[0];
  const center: [number, number] = selected ? [selected.lat, selected.lng] : [-23.5505, -46.6333];

  return (
    <section className="map-layout">
      <aside className="map-list">
        <div>
          <h2>Unidades de Saúde</h2>
          <p>Encontre a unidade mais próxima de você</p>
        </div>
        <div className="map-cards">
          {units.map((unit) => (
            <button
              className={`map-unit-card ${unit.id === selectedId ? 'active' : ''}`}
              key={unit.id}
              type="button"
              onClick={() => setSelectedId(unit.id)}
            >
              <div>
                <strong>{unit.name}</strong>
                <span>{unit.type}</span>
              </div>
              <StatusBadge status={unit.status} />
              <small>{unit.distance_km.toFixed(1)} km</small>
              <p>{unit.address}</p>
              <small>{unit.hours} · {unit.phone}</small>
            </button>
          ))}
        </div>
      </aside>

      <div className="map-panel">
        <MapContainer center={center} zoom={14} className="leaflet-map" zoomControl={false}>
          <TileLayer
            attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>'
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
          />
          <ZoomControl position="bottomright" />
        </MapContainer>
      </div>
    </section>
  );
}

```

```

    {units.map((unit) => (
      <Marker
        eventHandlers={{ click: () => setSelectedId(unit.id) }}
        icon={markerIcon}
        key={unit.id}
        position={[unit.lat, unit.lng]}
      >
        <Popup>
          <div className="map-popup">
            <strong>{unit.name}</strong>
            <span>{unit.type}</span>
            <p>{unit.address}</p>
            <a
              href={`https://www.google.com/maps/dir/?api=1&destination=${unit.lat},${unit.lng}`}
              target="_blank"
              rel="noreferrer"
            >
              <Navigation size={16} />
              Como Chegar
            </a>
          </div>
        </Popup>
      </Marker>
    )
    )}
  </MapContainer>

  {selected && (
    <article className="selected-map-card">
      <div>
        <strong>{selected.name}</strong>
        <span>{selected.type}</span>
      </div>
      <p>{selected.address}</p>
      <div className="chip-row">
        <span className="chip">{selected.status}</span>
        <span className="chip">{selected.distance_km.toFixed(1)} km</span>
        <span className="chip">{selected.phone}</span>
      </div>
      <a
        className="primary-action map-route"
        href={`https://www.google.com/maps/dir/?api=1&destination=${selected.lat},${selected.lng}`}
        target="_blank"
        rel="noreferrer"
      >
        <Navigation size={18} />
        Como Chegar
      </a>
    </article>
  )}
</div>
</section>
);
}

function ProfilePage({ profile, onSave }: { profile: PatientProfile; onSave: () => Promise<void> }) {
  const { refresh, user } = useAuth();
  const [form, setForm] = useState({
    cpf: profile.cpf || '',
    birthDate: profile.birth_date || '',
    phone: profile.phone || '',
    susCard: profile.sus_card || '',
    address: profile.address || '',
    emergencyContact: profile.emergency_contact || '',
  });
  const [message, setMessage] = useState('');
  const [avatarPreview, setAvatarPreview] = useState(user?.avatar || '');
  const [avatarBusy, setAvatarBusy] = useState(false);

  async function submit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setMessage('');
    try {
      await api('/profile', { method: 'PUT', body: form });
      setMessage('Cadastro atualizado com sucesso.');
      await onSave();
    } catch (error) {
      setMessage(error instanceof Error ? error.message : 'Falha ao salvar cadastro.');

```

```

        setMessage('A foto deve ter no maximo 2 MB.');
```

```

        return;
    }

    setAvatarBusy(true);
    try {
        const imageData = await fileToDataURL(file);
        setAvatarPreview(imageData);
        const payload = await api<{ user: User }>('/auth/avator', { method: 'POST', body: { imageData } });
        setAvatarPreview(payload.user.avatar);
        await refresh();
        setMessage('Foto atualizada com sucesso.');
```

```

    } catch (error) {
        setMessage(error instanceof Error ? error.message : 'Falha ao enviar foto.');
```

```

    } finally {
        setAvatarBusy(false);
        event.target.value = '';
    }
}

return (
    <form className="form-grid-panel" onSubmit={submit}>
        <SectionHeader icon={<UserPlus />} title="Cadastro do cidadão" />
        <div className="photo-uploader full-span">
            <Avatar label={avatarPreview || user?.name || 'SC'} />

```

```

// Linhas 563-809
        <div>
            <strong>Foto do perfil</strong>
            <span>Use uma imagem PNG, JPG ou WebP de ate 2 MB.</span>
        </div>
        <label className="upload-button">
            {avatarBusy ? 'Enviando...' : 'Anexar foto'}
            <input type="file" accept="image/png,image/jpeg,image/webp" onChange={uploadAvatar} disabled={avatarBusy} />
        </label>
    </div>
    <label>
        CPF
        <input value={form.cpf} onChange={(event) => setForm({ ...form, cpf: event.target.value })} />
    </label>
    <label>
        Data de nascimento
        <input
            type="date"
            value={form.birthDate}
            onChange={(event) => setForm({ ...form, birthDate: event.target.value })}
        />
    </label>
    <label>
        Telefone
        <input value={form.phone} onChange={(event) => setForm({ ...form, phone: event.target.value })} />
    </label>
    <label>
        Cartão SUS
        <input value={form.susCard} onChange={(event) => setForm({ ...form, susCard: event.target.value })} />
    </label>
    <label className="full-span">
        Endereço
        <input value={form.address} onChange={(event) => setForm({ ...form, address: event.target.value })} />
    </label>
    <label className="full-span">
        Contato de emergência
        <input
            value={form.emergencyContact}
            onChange={(event) => setForm({ ...form, emergencyContact: event.target.value })}
        />
    </label>
    {message && <div className="form-message full-span">{message}</div>}
    <button className="primary-action" type="submit">
        <Save size={18} />
        Salvar cadastro
    </button>
</form>
);
}

function RecordsPage({
    exams,
    onCreate,
    records,
}): {
    exams: DashboardPayload['exams'];
    onCreate: () => Promise<void>;
    records: RecordItem[];
} {
    const [category, setCategory] = useState('Histórico');
    const [title, setTitle] = useState('Nova evolução clínica');
    const [description, setDescription] = useState('Paciente relata melhora após orientações da equipe.');
```

```

    const [message, setMessage] = useState('');

    async function submit(event: React.FormEvent<HTMLFormElement>) {

```

```

    event.preventDefault();
    setMessage('');
    try {
      await api('/records', { method: 'POST', body: { category, title, description } });
      setMessage('Registro adicionado ao prontuário.');
```

await onCreate();

```
    } catch (error) {
      setMessage(error instanceof Error ? error.message : 'Falha ao registrar prontuário.');
```

}

```
  }
}

return (
  <section className="content-grid">
    <div className="timeline">
      <SectionHeader icon={<ClipboardList />} title="Prontuários" />
      {[...records, ...exams]
        .sort((a, b) => Date.parse('requested_at' in b ? b.requested_at : b.created_at) - Date.parse('requested_at'...
          .map((item) => (
            <div className="timeline-item" key={item.id}>
              <span className="timeline-dot" />
              <div>
                <strong>{item.title}</strong>
                <p>{'status' in item ? `${item.status} em ${item.unit}` : item.description}</p>
                <small>{formatDate('requested_at' in item ? item.requested_at : item.created_at)}</small>
              </div>
            </div>
          )))
    </div>
    <form className="action-panel" onSubmit={submit}>
      <SectionHeader icon={<Plus />} title="Adicionar registro" />
      <label>
        Categoria
        <input value={category} onChange={(event) => setCategory(event.target.value)} required />
      </label>
      <label>
        Título
        <input value={title} onChange={(event) => setTitle(event.target.value)} required />
      </label>
      <label>
        Descrição
        <textarea value={description} onChange={(event) => setDescription(event.target.value)} required />
      </label>
      {message} && <div className="form-message">{message}</div>
      <button className="primary-action" type="submit">
        <FileText size={18} />
        Salvar registro
      </button>
    </form>
  </section>
);
}

function TriagePage({ onCreate, triage }: { onCreate: () => Promise<void>; triage: TriageCase[] }) {
  const [symptoms, setSymptoms] = useState('Febre, dor no corpo e tosse há dois dias.');
```

const [riskLevel, setRiskLevel] = useState<TriageRisk>('medium');

const [message, setMessage] = useState('');

```

  async function submit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setMessage('');
    try {
      await api('/triage', { method: 'POST', body: { symptoms, riskLevel } });
      setMessage('Triagem registrada e enviada para a equipe.');
```

await onCreate();

```
    } catch (error) {
      setMessage(error instanceof Error ? error.message : 'Falha ao registrar triagem.');
```

}

```
  }
}

return (
  <section className="content-grid">
    <div className="stack">
      <SectionHeader icon={<Stethoscope />} title="Triagens recentes" />
      <div className="card-list">
        {triage.map((item) => (
          <article className="appointment-card" key={item.id}>
            <span className="icon-tile">
              <Activity size={20} />
            </span>
            <div>
              <div className="card-title-line">
                <strong>{riskLabel(item.risk_level)}</strong>
                <StatusBadge status={item.status} />
              </div>
              <p>{item.symptoms}</p>
              <small>{item.recommendation}</small>
            </div>
          </article>
        )))
      </div>
    </div>
  </div>
);

```

```

    <form className="action-panel" onSubmit={submit}>
      <SectionHeader icon={<Plus />} title="Nova triagem" />
      <label>
        Sintomas
        <textarea value={symptoms} onChange={(event) => setSymptoms(event.target.value)} required />
      </label>
      <label>
        Prioridade percebida
        <select value={riskLevel} onChange={(event) => setRiskLevel(event.target.value as TriageRisk)}>
          <option value="low">Baixa</option>
          <option value="medium">Média</option>
          <option value="high">Alta</option>
          <option value="critical">Emergência</option>
        </select>
      </label>
      {message && <div className="form-message">{message}</div>}
      <button className="primary-action" type="submit">
        <Stethoscope size={18} />
        Enviar triagem
      </button>
    </form>
  </section>
);
}

function QueuePage({ onCreate, queue, units }: { onCreate: () => Promise<void>; queue: QueueEntry[]; units: Unit[] ...
const [unitId, setUnitId] = useState(units[0]?.id || '');
const [service, setService] = useState('Clínica geral');
const [message, setMessage] = useState('');

async function submit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  setMessage('');
  try {
    await api('/queue', { method: 'POST', body: { unitId, service } });
    setMessage('Entrada adicionada à fila digital.');
    await onCreate();
  } catch (error) {
    setMessage(error instanceof Error ? error.message : 'Falha ao entrar na fila.');
```

```

// Linhas 810-1011
const [announcement, setAnnouncement] = useState({ title: '', body: '', audience: 'all' });

const load = () => {
  setError('');
  return api<AdminPayload>('/admin/overview')
    .then(setData)
    .catch((err) => setError(err instanceof Error ? err.message : 'Erro ao carregar administração.'));
};

useEffect(() => {
  void load();
}, []);

async function mutate(path: string, body: Record<string, unknown>, successMessage: string) {
  setError('');
  setNotice('');
  try {
    await api(path, { method: 'PATCH', body });
    setNotice(successMessage);
    await load();
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Não foi possível concluir a operação.');
```

```

  }
}

async function updateAppointment(id: string, status: AppointmentStatus) {
  await mutate(`/admin/appointments/${id}`, { status }, 'Agendamento atualizado.');
```

```

}

async function updateTriage(id: string, status: TriageStatus) {
  await mutate(`/admin/triage/${id}`, { status }, 'Triagem atualizada.');
```

```

}

async function updateQueue(id: string, status: QueueStatus) {
  await mutate(`/admin/queue/${id}`, { status }, 'Fila atualizada.');
```

```

}

async function updateIntegration(id: string, status: Integration['status']) {
  await mutate(`/admin/integrations/${id}`, { status }, 'Integração atualizada.');
```

```

}

async function updateTicket(id: string, status: TicketStatus) {
  await mutate(`/admin/tickets/${id}`, { status }, 'Chamado atualizado.');
```

```

}

async function updateUserRole(id: string, role: User['role']) {
  await mutate(`/admin/users/${id}/role`, { role }, 'Permissão atualizada.');
```

```

}

async function publishAnnouncement(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  setError('');
  setNotice('');
  try {
    await api('/admin/announcements', { method: 'POST', body: announcement });
    setAnnouncement({ title: '', body: '', audience: 'all' });
    setNotice('Aviso publicado para o portal.');
```

```

    await load();
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Não foi possível publicar o aviso.');
```

```

  }
}

function exportAdminReport() {
  if (!data) return;
  const rows = [
    ['categoria', 'nome', 'detalhe', 'status'],
    ...data.appointments.map((item) => ['agendamento', item.user_name || '', item.specialty, item.status]),
    ...data.triage.map((item) => ['triagem', item.user_name || '', item.symptoms, item.status]),
    ...data.queue.map((item) => ['fila', item.user_name || '', item.service, item.status]),
    ...data.tickets.map((item) => ['chamado', item.user_name || '', item.subject, item.status]),
  ];
  const csv = rows.map((row) => row.map((cell) => `${String(cell).replaceAll('"', '""')}").join(';')).join('\n');
  const url = URL.createObjectURL(new Blob([`ufeff${csv}`], { type: 'text/csv;charset=utf-8' }));
  const link = document.createElement('a');
  link.href = url;
  link.download = `saudeconnect-relatorio-${new Date().toISOString().slice(0, 10)}.csv`;
  link.click();
  URL.revokeObjectURL(url);
}

const normalizedQuery = query.trim().toLocaleLowerCase('pt-BR');
const matches = (...values: Array<string | number | undefined | null>) =>
  !normalizedQuery || values.some((value) => String(value || '').toLocaleLowerCase('pt-BR').includes(normalizedQuer...
const networkHealth = data?.integrations.length
  ? Math.round(data.integrations.reduce(
    (total, item) => total + (item.status === 'online' ? 100 : item.status === 'degraded' ? 70 : 20),
    0,
  ) / data.integrations.length)

```

```

: 0;
const visibleAppointments = data?.appointments.filter((item) =>
  matches(item.user_name, item.user_email, item.specialty, item.unit_name, item.status),
) || [];
const visibleTriage = data?.triage.filter((item) => matches(item.user_name, item.symptoms, item.status)) || [];
const visibleQueue = data?.queue.filter((item) => matches(item.user_name, item.service, item.unit_name, item.status...
const visibleUsers = data?.users.filter((item) => matches(item.name, item.email, item.role, item.provider)) || [];
const visibleTickets = data?.tickets.filter((item) => matches(item.user_name, item.subject, item.message, item.stat...

if (!data) return <DashboardShell title="Administração">{error ? <EmptyState message={error} /> : <PanelLoader />}<...

return (
  <DashboardShell title="Administração" subtitle="Operação, demanda, triagem, fila e integrações em tempo real.">
    <section className="admin-toolbar" aria-label="Ferramentas administrativas">
      <label className="admin-search">
        <Search size={18} />
        <input
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Buscar paciente, unidade, serviço ou status"
        />
      </label>
      <button className="admin-tool-button" type="button" onClick={() => void load()} title="Atualizar dados">
        <RefreshCw size={18} />
        Atualizar
      </button>
      <button className="admin-tool-button primary" type="button" onClick={exportAdminReport}>
        <Download size={18} />
        Exportar CSV
      </button>
    </section>

    {(notice || error) && <div className={`admin-feedback ${error ? 'error' : ''}`>{error || notice}</div>}

    <section className="metrics-grid">
      <MetricCard icon={UsersRound} label="Usuários" value={data.overview.users} tone="blue" />
      <MetricCard icon={CalendarDays} label="Agendamentos" value={data.overview.appointments} tone="green" />
      <MetricCard icon={Stethoscope} label="Triagens abertas" value={data.overview.triageWaiting} tone="orange"...
      <MetricCard icon={ListChecks} label="Fila aguardando" value={data.overview.queueWaiting} tone="purple" />
    </section>

    <section className="admin-overview-grid">
      <article className="network-card">
        <div className="network-card-header">
          <div>
            <span>Rede conectada</span>
            <small>Saúde dos serviços integrados</small>
          </div>
          <strong>{networkHealth}%</strong>
        </div>
        <div className="network-bars" aria-label={`Saúde da rede: ${networkHealth}%`} >
          {data.integrations.map((integration, index) => (
            <i
              className={`network-bar status-${integration.status}`}
              key={integration.id}
              style={{ height: `${48 + index * 11}px` }}
              title={`${integration.name}: ${integration.status}`}
            />
          ))}
        </div>
        <div className="network-summary">
          <div><Hospital size={18} /><strong>{data.overview.users}</strong><span>usuários</span></div>
          <div><CalendarDays size={18} /><strong>{data.overview.appointments}</strong><span>agendamentos</span></div>
          <div><Wrench size={18} /><strong>{data.overview.openTickets}</strong><span>chamados</span></div>
        </div>
      </article>

      <form className="announcement-form" onSubmit={publishAnnouncement}>
        <SectionHeader icon={Megaphone} title="Publicar aviso" />
        <label>
          Título
          <input
            value={announcement.title}
            onChange={(event) => setAnnouncement((current) => ({ ...current, title: event.target.value })})
            placeholder="Ex.: Campanha de vacinação"
            required
          />
        </label>
        <label>
          Mensagem
          <textarea
            value={announcement.body}
            onChange={(event) => setAnnouncement((current) => ({ ...current, body: event.target.value })})
            placeholder="Escreva o aviso que aparecerá no portal"
            required
          />
        </label>
        <label>
          Público
          <select
            value={announcement.audience}

```



```

        onChange={(event) => setAnnouncement((current) => ({ ...current, audience: event.target.value })))}
      >
      <option value="all">Todos</option>
      <option value="users">Usuários</option>
      <option value="admins">Administradores</option>
    </select>
  </label>
  <button className="primary-action" type="submit"><Megaphone size={18} />Publicar aviso</button>
</form>
</section>

<section className="admin-grid">
  <AdminTable title="Agenda da rede" icon={<CalendarDays />}>
    {visibleAppointments.map((appointment) => (
      <div className="admin-row" key={appointment.id}>
        <div>
          <strong>{appointment.user_name}</strong>
          <span>{appointment.specialty} - {appointment.unit_name}</span>
        </div>
        <small>{formatDate(appointment.scheduled_at)}</small>
        <select
// Linhas 1012-1237
          value={appointment.status}
          onChange={(event) => void updateAppointment(appointment.id, event.target.value as AppointmentStatus)}
        >
          <option value="pending">Pendente</option>
          <option value="confirmed">Confirmado</option>
          <option value="completed">Concluído</option>
          <option value="cancelled">Cancelado</option>
        </select>
      </div>
    ))}
    {!visibleAppointments.length && <div className="admin-empty">Nenhum agendamento encontrado.</div>}
  </AdminTable>

  <AdminTable title="Triagem" icon={<Stethoscope />} compact>
    {visibleTriage.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>{item.user_name}</strong>
          <span>{riskLabel(item.risk_level)} - {item.symptoms}</span>
        </div>
        <select value={item.status} onChange={(event) => void updateTriage(item.id, event.target.value as Triag...
          <option value="waiting">Aguardando</option>
          <option value="in_service">Em atendimento</option>
          <option value="resolved">Resolvida</option>
        </select>
      </div>
    ))}
    {!visibleTriage.length && <div className="admin-empty">Nenhuma triagem encontrada.</div>}
  </AdminTable>
</section>

<section className="admin-grid">
  <AdminTable title="Fila" icon={<ListChecks />}>
    {visibleQueue.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>#{item.position} - {item.user_name}</strong>
          <span>{item.service} - {item.unit_name}</span>
        </div>
        <small>{item.estimated_minutes} min</small>
        <select value={item.status} onChange={(event) => void updateQueue(item.id, event.target.value as QueueS...
          <option value="waiting">Aguardando</option>
          <option value="called">Chamado</option>
          <option value="done">Finalizado</option>
          <option value="cancelled">Cancelado</option>
        </select>
      </div>
    ))}
    {!visibleQueue.length && <div className="admin-empty">Nenhuma entrada de fila encontrada.</div>}
  </AdminTable>

  <AdminTable title="Integrações" icon={<Wifi />} compact>
    {data.integrations.map((integration) => (
      <div className="admin-row" key={integration.id}>
        <div>
          <strong>{integration.name}</strong>
          <span>{integration.latency_ms} ms - última sync {formatDate(integration.last_sync, false)}</span>
        </div>
        <StatusBadge status={integration.status} />
        <select
          value={integration.status}
          onChange={(event) => void updateIntegration(integration.id, event.target.value as Integration['status...
        >
          <option value="online">Online</option>
          <option value="degraded">Instável</option>
          <option value="offline">Offline</option>
        </select>
      </div>
    ))}
  </AdminTable>

```

```

    )))
  </AdminTable>
</section>

<section className="admin-grid">
  <AdminTable title="Usuários e permissões" icon={<UserCog />}>
    {visibleUsers.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>{item.name}</strong>
          <span>{item.email} - acesso {item.provider}</span>
        </div>
        <small>{item.last_login ? formatDate(item.last_login, false) : 'Sem acesso recente'}</small>
        <select value={item.role} onChange={(event) => void updateUserRole(item.id, event.target.value as User[...]}>
          <option value="user">Usuário</option>
          <option value="admin">Administrador</option>
        </select>
      </div>
    ))}
    {!visibleUsers.length && <div className="admin-empty">Nenhum usuário encontrado.</div>}
  </AdminTable>

  <AdminTable title="Chamados de suporte" icon={<ClipboardList />} compact>
    {visibleTickets.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>{item.subject}</strong>
          <span>{item.user_name} - {item.message}</span>
        </div>
        <StatusBadge status={item.priority === 'high' ? 'danger' : item.priority === 'medium' ? 'warn' : 'info'}...>
        <select value={item.status} onChange={(event) => void updateTicket(item.id, event.target.value as Ticket[...]}>
          <option value="open">Aberto</option>
          <option value="in_review">Em análise</option>
          <option value="resolved">Resolvido</option>
        </select>
      </div>
    ))}
    {!visibleTickets.length && <div className="admin-empty">Nenhum chamado encontrado.</div>}
  </AdminTable>
</section>

<section className="admin-grid">
  <AdminTable title="Avisos publicados" icon={<Megaphone />} compact>
    {data.announcements.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>{item.title}</strong>
          <span>{item.body}</span>
        </div>
        <small>{item.audience} - {formatDate(item.published_at, false)}</small>
      </div>
    ))}
  </AdminTable>

  <AdminTable title="Atividade recente" icon={<ShieldCheck />} compact>
    {data.auditLogs.map((item) => (
      <div className="admin-row" key={item.id}>
        <div>
          <strong>{item.actor_name || 'Sistema'}</strong>
          <span>{item.action} em {item.entity}</span>
        </div>
        <small>{formatDate(item.created_at, false)}</small>
      </div>
    ))}
  </AdminTable>
</section>
</DashboardShell>
);
}

```

```

function DashboardShell({ children, title, subtitle }: { children: React.ReactNode; title: string; subtitle?: string ...
const { user, logout } = useAuth();
const homePath = user?.role === 'admin' ? '/admin' : '/app';

```

```

return (
  <div className="app-shell">
    <aside className="sidebar">
      <BrandLockup small />
      <nav>
        <NavLink end to={homePath}>
          <LayoutDashboard size={18} />
          Início
        </NavLink>
        <NavLink to="/app/mapa">
          <Map size={18} />
          Mapa
        </NavLink>
        <NavLink to="/app/cadastro">
          <UserPlus size={18} />
          Cadastro
        </NavLink>

```

```

    <NavLink to="/app/prontuarios">
      <FileText size={18} />
      Prontuários
    </NavLink>
    <NavLink to="/app/triagem">
      <Stethoscope size={18} />
      Triagem
    </NavLink>
    <NavLink to="/app/fila">
      <ListChecks size={18} />
      Fila
    </NavLink>
  </nav>
</aside>

<main className="workspace">
  <header className="topbar">
    <div>
      <p className="eyebrow">{user?.role === 'admin' ? 'Gestão da rede' : 'Sistema Integrado'}</p>
      <h1>{title}</h1>
      {subtitle && <p>{subtitle}</p>}
    </div>
    <div className="user-pill">
      <Bell size={18} />
      <Avatar label={user?.avatar || user?.name || 'SC'} />
      <span>{user?.name}</span>
      <button type="button" onClick={logout} aria-label="Sair">
        <LogOut size={18} />
      </button>
    </div>
  </header>
  {children}
</main>
</div>
);
}

function BrandLockup({ small = false }: { small?: boolean }) {
  return (
    <div className={`brand-lockup ${small ? 'small' : ''}`>
      <div className="brand-mark">
        <HeartPulse size={small ? 22 : 26} />
      </div>
      <div>
        <strong>SaúdeConnect</strong>
        <span>Sistema Integrado</span>
      </div>
    </div>
  );
}

function AppointmentForm({ units, onCreated }: { units: Unit[]; onCreated: () => Promise<void> }) {
  const [unitId, setUnitId] = useState(units[0]?.id || '');
  const [specialty, setSpecialty] = useState('Clínica geral');
  const [scheduledAt, setScheduledAt] = useState(nextDateInput());
  const [reason, setReason] = useState('Preciso de acompanhamento de rotina.');
```

const [message, setMessage] = useState('');

```

  async function submit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setMessage('');
    try {
      await api('/appointments', { method: 'POST', body: { unitId, specialty, scheduledAt, reason } });
      setMessage('Pedido enviado para confirmação.');
```

await onCreated();

```

    } catch (error) {

// Linhas 1238-1471
    setMessage(error instanceof Error ? error.message : 'Falha ao solicitar agendamento.');
```

}

```

  }

  return (
    <form className="action-panel" onSubmit={submit}>
      <SectionHeader icon={Plus} title="Solicitar agendamento" />
      <label>
        Unidade
        <select value={unitId} onChange={(event) => setUnitId(event.target.value)} required>
          {units.map((unit) => (
            <option key={unit.id} value={unit.id}>
              {unit.name}
            </option>
          ))}
        </select>
      </label>
      <label>
        Especialidade
        <input value={specialty} onChange={(event) => setSpecialty(event.target.value)} required />
      </label>
      <label>
        Data desejada
```

```

        <input type="datetime-local" value={scheduledAt} onChange={(event) => setScheduledAt(event.target.value)} req...
      </label>
      <label>
        Motivo
        <textarea value={reason} onChange={(event) => setReason(event.target.value)} required />
      </label>
      {message && <div className="form-message">{message}</div>}
      <button className="primary-action" type="submit">
        <CalendarDays size={18} />
        Enviar pedido
      </button>
    </form>
  );
}

function AppointmentCard({ appointment }: { appointment: Appointment }) {
  return (
    <article className="appointment-card">
      <div>
        <span className="icon-tile">
          <CalendarDays size={20} />
        </span>
      </div>
      <div>
        <div className="card-title-line">
          <strong>{appointment.specialty}</strong>
          <StatusBadge status={appointment.status} />
        </div>
        <p>{appointment.unit_name} - {appointment.professional}</p>
        <small>{formatDate(appointment.scheduled_at)} - {appointment.reason}</small>
      </div>
    </article>
  );
}

function MetricCard({ icon, label, value, tone }: { icon: React.ReactNode; label: string; value: number; tone: string...
  return (
    <article className={`metric-card tone-${tone}`}>
      <span>{icon}</span>
      <div>
        <strong>{value}</strong>
        <p>{label}</p>
      </div>
    </article>
  );
}

function SectionHeader({ icon, title, action }: { icon: React.ReactNode; title: string; action?: string }) {
  return (
    <div className="section-header">
      <div>
        <span>{icon}</span>
        <h2>{title}</h2>
      </div>
      {action && <small>{action}</small>}
    </div>
  );
}

function AdminTable({
  children,
  compact = false,
  icon,
  title,
}: {
  children: React.ReactNode;
  compact?: boolean;
  icon: React.ReactNode;
  title: string;
}) {
  return (
    <section className={`admin-panel ${compact ? 'compact' : ''}`}>
      <SectionHeader icon={icon} title={title} />
      <div className="admin-list">{children}</div>
    </section>
  );
}

function StatusBadge({ status }: { status: string }) {
  const labels: Record<string, string> = {
    pending: 'Pendente',
    confirmed: 'Confirmado',
    completed: 'Concluído',
    cancelled: 'Cancelado',
    waiting: 'Aguardando',
    in_service: 'Em atendimento',
    resolved: 'Resolvido',
    called: 'Chamado',
    done: 'Finalizado',
    online: 'Online',
    degraded: 'Instável',
  };

```

```

    offline: 'Offline',
    Aberto: 'Aberto',
    'Coleta até 16h': 'Coleta até 16h',
    'Plantão reduzido': 'Plantão reduzido',
  };
  return <span className={`status-badge status-${statusClass(status)}`}>{labels[status] || status}</span>;
}

function statusClass(status: string) {
  if (['confirmed', 'resolved', 'online', 'completed', 'done', 'called', 'Aberto'].includes(status)) return 'ok';
  if (['pending', 'waiting', 'degraded', 'Coleta até 16h', 'Plantão reduzido'].includes(status)) return 'warn';
  if (['cancelled', 'offline', 'critical'].includes(status)) return 'danger';
  if (status === 'in_service') return 'info';
  return 'default';
}

function Avatar({ label }: { label: string }) {
  const initials = useMemo(
    () =>
      label
        .split(/\s+/)
        .filter(Boolean)
        .slice(0, 2)
        .map((part) => part[0])
        .join('')
        .toUpperCase(),
    [label],
  );

  if (label.startsWith('/uploads/') || /^https?:\/\//.test(label)) {
    return <img className="avatar avatar-image" src={mediaUrl(label)} alt="Foto do perfil" />;
  }

  return <span className="avatar">{initials}</span>;
}

function fileToDataURL(file: File) {
  return new Promise<string>((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => resolve(String(reader.result));
    reader.onerror = () => reject(new Error('Nao foi possivel ler a imagem.'));
    reader.readAsDataURL(file);
  });
}

function FullPageLoader({ label = 'Carregando...' }: { label?: string }) {
  return (
    <div className="full-loader">
      <HeartPulse size={32} />
      <span>{label}</span>
    </div>
  );
}

function PanelLoader() {
  return (
    <div className="panel-loader">
      <RefreshCw className="spin" size={24} />
      <span>Sincronizando dados...</span>
    </div>
  );
}

function EmptyState({ message }: { message: string }) {
  return (
    <div className="empty-state">
      <AlertTriangle size={24} />
      <strong>{message}</strong>
    </div>
  );
}

function pageTitle(page: PortalPageName) {
  return {
    home: 'Início',
    map: 'Unidades de Saúde',
    profile: 'Cadastro',
    records: 'Prontuários',
    triage: 'Triagem',
    queue: 'Fila',
  }[page];
}

function pageSubtitle(page: PortalPageName) {
  return {
    home: 'Resumo do atendimento, próximos passos e avisos importantes.',
    map: 'Encontre a unidade mais próxima de você',
    profile: 'Mantenha seus dados atualizados para agilizar o atendimento.',
    records: 'Histórico clínico, exames e registros de cuidado.',
    triage: 'Informe sintomas e acompanhe a classificação de prioridade.',
    queue: 'Acompanhe sua posição e entre na fila digital das unidades.',
  };
}

```

```
    }[page];
  }

  function riskLabel(risk: TriageRisk) {
    return {
      low: 'Prioridade verde',
      medium: 'Prioridade amarela',
      high: 'Prioridade laranja',
      critical: 'Prioridade vermelha',
    }[risk];
  }

  function formatDate(value: string, includeTime = true) {
    return new Intl.DateTimeFormat('pt-BR', {
      day: '2-digit',
      month: 'short',
      year: 'numeric',
      ...(includeTime ? { hour: '2-digit', minute: '2-digit' } : {}),
    }).format(new Date(value));
  }

  function nextDateInput() {
    const date = new Date(Date.now() + 86400000 * 3);
    date.setMinutes(0, 0, 0);
    return date.toISOString().slice(0, 16);
  }

  export default App;
```

Arquivo: src/index.css

Folha de estilos global: variáveis CSS, layout responsivo, auth shell, sidebar, cards, mapa, formularios, painel admin e animacoes.

Total: 1639 linhas | Linguagem: .css

```
// Linhas 1-548
:root {
  --bg: #f5f7fb;
  --surface: #ffffff;
  --surface-strong: #eef6f7;
  --text: #1f2937;
  --muted: #697586;
  --border: #dde5ee;
  --line: #edf1f5;
  --teal: #0f8b8d;
  --teal-dark: #0b5f61;
  --blue: #2563eb;
  --green: #159947;
  --orange: #e07a2f;
  --purple: #7152b7;
  --red: #c2413a;
  --amber-soft: #fff4dc;
  --blue-soft: #eaf1ff;
  --green-soft: #e8f7ee;
  --orange-soft: #fff0e6;
  --purple-soft: #f0ebff;
  --shadow: 0 18px 60px rgba(31, 41, 55, 0.1);
  --shadow-soft: 0 12px 32px rgba(31, 41, 55, 0.08);
  font-family:
    Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: var(--text);
  background: var(--bg);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
  overflow-x: hidden;
}

button,
input,
select,
textarea {
  font: inherit;
}

button,
a,
select {
  -webkit-tap-highlight-color: transparent;
}

button {
  cursor: pointer;
}

a {
  color: inherit;
  text-decoration: none;
}

h1,
h2,
h3,
p {
  margin: 0;
}

.auth-shell {
  display: grid;
  grid-template-columns: minmax(0, 1.05fr) minmax(420px, 0.95fr);
  min-height: 100vh;
  background:
    radial-gradient(circle at 8% 15%, rgba(15, 139, 141, 0.18), transparent 28%),
    linear-gradient(135deg, #f7fbfd 0%, #eef5f1 48%, #f8f4ee 100%);
}

.auth-visual {
```

```
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 40px;
    padding: clamp(28px, 5vw, 72px);
}

.brand-lockup {
  display: flex;
  align-items: center;
  gap: 14px;
}

.brand-lockup.small {
  padding: 4px 0 20px;
}

.brand-mark {
  display: grid;
  place-items: center;
  width: 52px;
  height: 52px;
  border-radius: 16px;
  color: white;
  background: linear-gradient(135deg, var(--teal), #2659a8);
  box-shadow: 0 16px 34px rgba(15, 139, 141, 0.26);
}

.brand-lockup.strong {
  display: block;
  color: #10212b;
  font-size: 1.05rem;
}

.brand-lockup.span {
  display: block;
  color: var(--muted);
  font-size: 0.88rem;
}

.live-board {
  width: min(560px, 100%);
  padding: 28px;
  border: 1px solid rgba(255, 255, 255, 0.78);
  border-radius: 28px;
  background: rgba(255, 255, 255, 0.68);
  box-shadow: var(--shadow);
  backdrop-filter: blur(22px);
}

.live-board-header,
.unit-topline,
.card-title-line,
.section-header,
.section-header > div,
.user-pill,
.admin-row {
  display: flex;
  align-items: center;
}

.live-board-header {
  justify-content: space-between;
  color: var(--muted);
}

.live-board-header.strong {
  color: var(--teal-dark);
  font-size: 2rem;
}

.pulse-line {
  display: grid;
  grid-template-columns: 0.7fr 1.4fr 0.9fr 1.1fr;
  gap: 10px;
  align-items: end;
  height: 90px;
  margin: 26px 0;
}

.pulse-line i {
  display: block;
  border-radius: 999px 999px 10px 10px;
  background: linear-gradient(180deg, #2f80ed, var(--teal));
}

.pulse-line i:nth-child(1) {
  height: 46px;
}

.pulse-line i:nth-child(2) {
```



```
    height: 82px;
}

.pulse-line i:nth-child(3) {
    height: 58px;
    background: linear-gradient(180deg, #f2a65a, #df6f3d);
}

.pulse-line i:nth-child(4) {
    height: 70px;
    background: linear-gradient(180deg, #7c5ac9, #2f80ed);
}

.mini-grid,
.metrics-grid,
.unit-grid,
.admin-grid,
.content-grid {
    display: grid;
    gap: 16px;
}

.mini-grid {
    grid-template-columns: repeat(3, 1fr);
}

.mini-grid div {
    padding: 16px;
    border: 1px solid var(--line);
    border-radius: 18px;
    background: rgba(255, 255, 255, 0.72);
}

.mini-grid svg {
    color: var(--teal);
}

.mini-grid strong {
    display: block;
    margin-top: 12px;
    font-size: 1.45rem;
}

.mini-grid span {
    color: var(--muted);
    font-size: 0.86rem;
}

.auth-copy {
    max-width: 720px;
}

.auth-copy h1 {
    color: #10212b;
    font-size: clamp(2.4rem, 7vw, 5.8rem);
    line-height: 0.94;
    letter-spacing: 0;
}

.auth-copy p {
    max-width: 620px;
    margin-top: 22px;
    color: #485769;
    font-size: 1.06rem;
    line-height: 1.65;
}

.auth-panel {
    display: grid;
    place-items: center;
    padding: 28px;
}

.auth-card,
.action-panel,
.admin-panel,
.appointment-card,
.unit-card,
.metric-card {
    border: 1px solid var(--border);
    background: var(--surface);
    box-shadow: var(--shadow-soft);
}

.auth-card {
    width: min(460px, 100%);
    padding: 26px;
    border-radius: 28px;
}

.segmented {
```

```

display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 8px;
padding: 6px;
border-radius: 16px;
background: #f1f5f9;
}

.segmented button,
.user-pill button {
border: 0;
background: transparent;
}

.segmented button {
display: flex;
align-items: center;
justify-content: center;
gap: 8px;
min-height: 44px;
border-radius: 12px;
color: var(--muted);
font-weight: 700;
}

.segmented button.active {
color: var(--teal-dark);
background: white;
box-shadow: 0 8px 20px rgba(31, 41, 55, 0.08);
}

.auth-form,
.action-panel,
.stack,
.admin-panel,
.admin-list,
.timeline,
.card-list {
display: flex;
flex-direction: column;
}

.auth-form {
gap: 18px;
margin-top: 26px;
}

.eyebrow {
color: var(--teal);
font-size: 0.78rem;
font-weight: 800;
letter-spacing: 0.08em;
text-transform: uppercase;
}

.auth-form h2,
.section-header h2 {
font-size: 1.35rem;
letter-spacing: 0;
}

label {
display: grid;
gap: 8px;
color: #344054;
font-size: 0.92rem;
font-weight: 700;
}

label span {
display: flex;
align-items: center;
gap: 10px;
min-height: 48px;
padding: 0 14px;
border: 1px solid var(--border);
border-radius: 14px;
background: #fbfdff;
color: var(--muted);
}

input,
select,
textarea {
width: 100%;
border: 1px solid var(--border);
border-radius: 14px;
background: #fbfdff;
color: var(--text);
outline: 0;
}

```

```

label span input {
  min-width: 0;
  border: 0;
  border-radius: 0;
  background: transparent;
}

select,
textarea,
.action-panel input {
  min-height: 46px;
  padding: 10px 12px;
}

textarea {
  min-height: 98px;
  resize: vertical;
}

input:focus,
select:focus,
textarea:focus,
label span:focus-within {
  border-color: rgba(15, 139, 141, 0.62);
  box-shadow: 0 0 0 4px rgba(15, 139, 141, 0.12);
}

.field-hint {
  display: block;
  margin-top: -10px;
  color: var(--muted);
  font-size: 0.78rem;
  line-height: 1.4;
}

.primary-action,
.google-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  min-height: 48px;
  border-radius: 14px;
  border: 0;
  font-weight: 800;
}

.primary-action {
  color: white;
  background: linear-gradient(135deg, var(--teal), #2f6dcc);
  box-shadow: 0 16px 28px rgba(15, 139, 141, 0.24);
}

.primary-action:disabled {
  cursor: not-allowed;
  opacity: 0.7;
}

.oauth-area {
  display: grid;
  gap: 8px;
  margin-top: 18px;
  text-align: center;
}

.google-button {
  border: 1px solid var(--border);
  color: #344054;
  background: white;
}

.google-button.disabled {
  pointer-events: none;
  color: var(--muted);
  background: #f8fafc;
}

.oauth-area small,
.section-header small,
.appointment-card small,
.admin-row small,
.audit-box small {
  color: var(--muted);
}

.form-message {
  padding: 12px 14px;
  border-radius: 14px;
  color: var(--teal-dark);
  background: #e6f5f3;
  font-weight: 700;
}

```

```
}

.form-message.error {
  color: var(--red);
  background: #fff1ef;
}

.app-shell {
  display: grid;
  grid-template-columns: 280px minmax(0, 1fr);
  min-height: 100vh;
  background: var(--bg);
  overflow-x: hidden;
}

.sidebar {
  position: sticky;
  top: 0;
  height: 100vh;
  padding: 24px;
  border-right: 1px solid var(--border);
  background: rgba(255, 255, 255, 0.88);
  backdrop-filter: blur(18px);
}

.sidebar nav {
  display: grid;
  gap: 8px;
}

.sidebar a {
  display: flex;
  align-items: center;
  gap: 12px;
  min-height: 46px;
  padding: 0 14px;
  border-radius: 14px;
  color: var(--muted);
  font-weight: 800;
}

.sidebar a.active,
.sidebar a:hover {
  color: var(--teal-dark);
  background: #eaf6f4;
}

.sidebar a.active {
  border: 1px solid rgba(15, 139, 141, 0.16);
  box-shadow: inset 4px 0 0 var(--teal);
}

.workspace {
  display: grid;
  gap: 22px;
  align-content: start;
  min-width: 0;
  padding: 28px;
}

.topbar {
  display: flex;
  justify-content: space-between;
  gap: 18px;
  align-items: flex-start;
  min-width: 0;
  padding: 26px;
  border: 1px solid var(--border);
  border-radius: 26px;
  background:
    linear-gradient(135deg, rgba(255, 255, 255, 0.92), rgba(238, 246, 247, 0.88)),
    var(--surface);
}

.topbar h1 {
  margin-top: 4px;
  font-size: clamp(2rem, 4vw, 3.6rem);
  line-height: 1;
  letter-spacing: 0;
  overflow-wrap: anywhere;
}

.topbar p:last-child {
  margin-top: 10px;
  color: var(--muted);
  overflow-wrap: anywhere;
}

.user-pill {
  gap: 10px;
  min-width: 0;
}
```

```

    min-height: 46px;
    padding: 6px 8px 6px 12px;
    border: 1px solid var(--border);
    border-radius: 999px;
    background: white;
    white-space: nowrap;
}

.user-pill strong {

// Linhas 549-1106
    overflow: hidden;
    min-width: 0;
    text-overflow: ellipsis;
}

.user-pill > svg {
    color: var(--orange);
}

.user-pill button {
    display: grid;
    place-items: center;
    width: 34px;
    height: 34px;
    border-radius: 999px;
    color: var(--muted);
}

.user-pill button:hover {
    color: var(--red);
    background: #fff1ef;
}

.avatar {
    display: inline-grid;
    place-items: center;
    width: 34px;
    height: 34px;
    border-radius: 12px;
    color: white;
    background: linear-gradient(135deg, var(--purple), var(--teal));
    font-size: 0.8rem;
    font-weight: 900;
}

.avatar-image {
    display: block;
    object-fit: cover;
    border: 2px solid white;
}

.photo-uploader {
    display: grid;
    grid-template-columns: auto minmax(0, 1fr) auto;
    align-items: center;
    gap: 16px;
    padding: 16px;
    border: 1px solid var(--border);
    border-radius: 18px;
    background: #fbfdff;
}

.photo-uploader .avatar {
    width: 58px;
    height: 58px;
    border-radius: 18px;
    font-size: 1rem;
}

.photo-uploader strong,
.photo-uploader span {
    display: block;
}

.photo-uploader span {
    margin-top: 4px;
    color: var(--muted);
    font-size: 0.84rem;
    font-weight: 500;
}

.upload-button {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-height: 42px;
    padding: 0 16px;
    border-radius: 12px;
    color: var(--teal-dark);
    background: #edf5f3;
}

```

```

    font-weight: 900;
    cursor: pointer;
}

.upload-button input {
  position: absolute;
  width: 1px;
  height: 1px;
  opacity: 0;
  pointer-events: none;
}

.metrics-grid {
  grid-template-columns: repeat(4, minmax(0, 1fr));
}

.metric-card {
  display: flex;
  align-items: center;
  gap: 16px;
  min-height: 118px;
  padding: 20px;
  border-radius: 22px;
}

.metric-card > span {
  display: grid;
  place-items: center;
  width: 48px;
  height: 48px;
  border-radius: 16px;
}

.metric-card strong {
  font-size: 2rem;
  line-height: 1;
}

.metric-card p {
  margin-top: 6px;
  color: var(--muted);
  font-weight: 700;
}

.tone-blue > span {
  color: var(--blue);
  background: var(--blue-soft);
}

.tone-green > span {
  color: var(--green);
  background: var(--green-soft);
}

.tone-orange > span {
  color: var(--orange);
  background: var(--orange-soft);
}

.tone-purple > span {
  color: var(--purple);
  background: var(--purple-soft);
}

.content-grid,
.admin-grid {
  grid-template-columns: minmax(0, 1.35fr) minmax(320px, 0.65fr);
}

.stack,
.action-panel,
.admin-panel,
.units-band {
  gap: 16px;
  min-width: 0;
}

.action-panel,
.admin-panel,
.units-band {
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: 24px;
  background: var(--surface);
  box-shadow: var(--shadow-soft);
}

.section-header {
  justify-content: space-between;
  gap: 14px;
}

```

```

.section-header > div {
  gap: 12px;
  min-width: 0;
}

.section-header span {
  display: grid;
  place-items: center;
  flex: 0 0 auto;
  width: 38px;
  height: 38px;
  border-radius: 14px;
  color: var(--teal-dark);
  background: #e6f5f3;
}

.card-list {
  gap: 12px;
}

.appointment-card {
  display: grid;
  grid-template-columns: auto minmax(0, 1fr);
  gap: 14px;
  padding: 18px;
  border-radius: 20px;
}

.icon-tile {
  display: grid;
  place-items: center;
  width: 44px;
  height: 44px;
  border-radius: 15px;
  color: var(--blue);
  background: var(--blue-soft);
}

.card-title-line {
  justify-content: space-between;
  gap: 12px;
}

.appointment-card p {
  margin-top: 7px;
  color: #3d4b5c;
}

.appointment-card small {
  display: block;
  margin-top: 8px;
}

.status-badge,
.chip {
  display: inline-flex;
  align-items: center;
  min-height: 28px;
  padding: 4px 9px;
  border-radius: 999px;
  font-size: 0.76rem;
  font-weight: 900;
}

.status-pending,
.status-open,
.status-degraded,
.status-warn {
  color: #9a5a00;
  background: var(--amber-soft);
}

.status-confirmed,
.status-resolved,
.status-online,
.status-completed,
.status-ok {
  color: #116b34;
  background: var(--green-soft);
}

.status-cancelled,
.status-offline,
.status-danger {
  color: var(--red);
  background: #fff1ef;
}

.status-in_review,
.status-info {
  color: #2356a0;
}

```

```

    background: var(--blue-soft);
}

.status-default {
  color: #344054;
  background: #f2f6f9;
}

.timeline {
  gap: 12px;
  padding: 18px;
  border: 1px solid var(--border);
  border-radius: 22px;
  background: var(--surface);
  box-shadow: var(--shadow-soft);
}

.timeline-item {
  position: relative;
  display: grid;
  grid-template-columns: 22px minmax(0, 1fr);
  gap: 12px;
  padding-bottom: 14px;
}

.timeline-item:not(:last-child)::before {
  content: "";
  position: absolute;
  left: 8px;
  top: 20px;
  bottom: -4px;
  width: 2px;
  background: var(--line);
}

.timeline-dot {
  position: relative;
  z-index: 1;
  width: 18px;
  height: 18px;
  margin-top: 3px;
  border: 4px solid #dff4f0;
  border-radius: 999px;
  background: var(--teal);
}

.timeline p {
  margin-top: 4px;
  color: var(--muted);
}

.timeline small {
  display: block;
  margin-top: 6px;
  color: var(--muted);
}

.units-band {
  display: grid;
}

.unit-grid {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

.unit-card {
  padding: 18px;
  border-radius: 20px;
}

.unit-topline {
  justify-content: space-between;
  gap: 12px;
}

.unit-topline span {
  color: var(--teal-dark);
  font-size: 0.78rem;
  font-weight: 900;
}

.unit-card p {
  margin-top: 10px;
  color: var(--muted);
}

.chip-row {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 14px;
}

```



```

}

.chip {
  color: #344054;
  background: #f2f6f9;
}

.admin-list {
  gap: 10px;
}

.admin-row {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto minmax(132px, auto);
  gap: 12px;
  padding: 14px;
  border: 1px solid var(--line);
  border-radius: 18px;
  background: #fbfdff;
}

.admin-row > div {
  min-width: 0;
}

.admin-row strong,
.admin-row span {
  display: block;
}

.admin-row span {
  overflow: hidden;
  color: var(--muted);
  font-size: 0.88rem;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.admin-row select {
  min-height: 38px;
}

.admin-row.user-row {
  grid-template-columns: auto minmax(0, 1fr);
}

.admin-panel.compact .admin-row {
  grid-template-columns: 1fr;
  align-items: stretch;
}

.admin-panel.compact .admin-row.user-row {
  grid-template-columns: auto minmax(0, 1fr);
  align-items: center;
}

.audit-box {
  display: grid;
  gap: 8px;
  padding: 14px;
  border-radius: 18px;
  background: #f8fafc;
}

.admin-toolbar {
  display: grid;
  grid-template-columns: minmax(260px, 1fr) auto auto;
  gap: 12px;
  align-items: center;
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 20px;
  background: var(--surface);
  box-shadow: var(--shadow-soft);
}

.admin-search {
  display: flex;
  align-items: center;
  gap: 10px;
  min-height: 46px;
  padding: 0 14px;
  border: 1px solid var(--border);
  border-radius: 14px;
  color: var(--muted);
  background: #fbfdff;
}

.admin-search input {
  min-width: 0;
  border: 0;
}

```

```
background: transparent;
}

.admin-tool-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 46px;
  padding: 0 16px;
  border: 1px solid var(--border);
  border-radius: 14px;
  color: var(--teal-dark);
  background: white;
  font-weight: 900;
}

.admin-tool-button.primary {
  border-color: transparent;
  color: white;
  background: linear-gradient(135deg, var(--teal), #2f70cf);
}

.admin-feedback {
  padding: 12px 16px;
  border: 1px solid #bde6ca;
  border-radius: 14px;
  color: #116b34;
  background: var(--green-soft);
  font-weight: 800;
}

.admin-feedback.error {
  border-color: #f2c8c4;
  color: var(--red);
  background: #fff1ef;
}

.admin-overview-grid {
  display: grid;
  grid-template-columns: minmax(0, 1.15fr) minmax(340px, 0.85fr);
  gap: 16px;
}

.network-card,
.announcement-form {
  min-width: 0;
  padding: 22px;
  border: 1px solid var(--border);
  border-radius: 24px;
  background: var(--surface);
  box-shadow: var(--shadow-soft);
}

.network-card {
  display: grid;
  gap: 20px;
}

.network-card-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
}

.network-card-header span,
.network-card-header small {
  display: block;
}

.network-card-header span {
  color: var(--muted);
  font-weight: 800;
}

.network-card-header small {
  margin-top: 4px;
  color: #8a95a4;
}

.network-card-header strong {
  color: var(--teal-dark);
  font-size: 2.25rem;
  line-height: 1;
}

.network-bars {
  display: grid;
  grid-template-columns: repeat(4, minmax(42px, 1fr));
  gap: 12px;
}
```

```

    align-items: end;
    min-height: 104px;
}

.network-bar {
  display: block;
  border-radius: 999px 999px 10px 10px;
  background: linear-gradient(180deg, #2f80ed, var(--teal));
  transform-origin: bottom;
}

.network-bar.status-degraded {
  background: linear-gradient(180deg, #f2a65a, #df6f3d);
}

.network-bar.status-offline {
  background: linear-gradient(180deg, #dc6b62, #b8322b);
}

.network-summary {
  display: grid;
}

// Linhas 1107-1641
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 12px;
}

.network-summary div {
  display: grid;
  gap: 5px;
  padding: 14px;
  border: 1px solid var(--line);
  border-radius: 16px;
  background: #fbfdff;
}

.network-summary svg {
  color: var(--teal);
}

.network-summary strong {
  font-size: 1.4rem;
}

.network-summary span {
  overflow: hidden;
  color: var(--muted);
  font-size: 0.8rem;
  text-overflow: ellipsis;
}

.announcement-form {
  display: grid;
  gap: 14px;
}

.announcement-form textarea {
  min-height: 92px;
}

.announcement-form .primary-action {
  width: fit-content;
  padding: 0 18px;
}

.admin-empty {
  padding: 18px;
  border: 1px dashed var(--border);
  border-radius: 16px;
  color: var(--muted);
  text-align: center;
  background: #fbfdff;
}

.map-layout {
  display: grid;
  grid-template-columns: minmax(320px, 0.42fr) minmax(0, 1fr);
  gap: 18px;
  min-height: 680px;
}

.map-list,
.map-panel,
.form-grid-panel,
.queue-card {
  border: 1px solid var(--border);
  background: var(--surface);
  box-shadow: var(--shadow-soft);
}

.map-list {

```

```
    display: flex;
    flex-direction: column;
    gap: 18px;
    padding: 20px;
    border-radius: 24px;
    min-width: 0;
  }

.map-list h2 {
  font-size: 1.45rem;
}

.map-list p {
  margin-top: 4px;
  color: var(--muted);
}

.map-cards {
  display: grid;
  gap: 12px;
  overflow: auto;
  padding-right: 4px;
}

.map-unit-card {
  display: grid;
  gap: 8px;
  padding: 16px;
  border: 1px solid var(--line);
  border-radius: 18px;
  background: #fbfdff;
  color: var(--text);
  text-align: left;
}

.map-unit-card.active {
  border-color: rgba(15, 139, 141, 0.55);
  background: #effaf8;
  box-shadow: 0 12px 24px rgba(15, 139, 141, 0.1);
}

.map-unit-card strong,
.selected-map-card strong {
  display: block;
}

.map-unit-card span,
.selected-map-card span {
  color: var(--muted);
  font-size: 0.9rem;
}

.map-unit-card small {
  color: var(--muted);
  font-weight: 800;
}

.map-panel {
  position: relative;
  min-height: 680px;
  overflow: hidden;
  border-radius: 24px;
}

.leaflet-map {
  width: 100%;
  height: 100%;
  min-height: 680px;
}

.health-marker {
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: rgba(15, 139, 141, 0.16);
}

.health-marker span {
  display: block;
  width: 18px;
  height: 18px;
  border: 4px solid white;
  border-radius: 999px;
  background: var(--teal);
  box-shadow: 0 8px 18px rgba(15, 139, 141, 0.32);
}

.map-popup {
  display: grid;
  gap: 6px;
  min-width: 190px;
}
```

```

}

.map-popup span,
.map-popup p {
  color: var(--muted);
}

.map-popup a {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  margin-top: 4px;
  color: var(--teal-dark);
  font-weight: 900;
}

.selected-map-card {
  position: absolute;
  left: 18px;
  bottom: 18px;
  z-index: 500;
  display: grid;
  gap: 12px;
  width: min(420px, calc(100% - 36px));
  padding: 18px;
  border: 1px solid var(--border);
  border-radius: 20px;
  background: rgba(255, 255, 255, 0.94);
  box-shadow: var(--shadow);
  backdrop-filter: blur(18px);
}

.map-route {
  width: fit-content;
  padding: 0 16px;
}

.form-grid-panel {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
  padding: 22px;
  border-radius: 24px;
}

.form-grid-panel .section-header,
.form-grid-panel .full-span {
  grid-column: 1 / -1;
}

.form-grid-panel button {
  width: fit-content;
  padding: 0 18px;
}

.queue-board {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 14px;
}

.queue-card {
  display: grid;
  gap: 10px;
  padding: 18px;
  border-radius: 20px;
}

.queue-card strong {
  display: block;
  font-size: 2rem;
  line-height: 1;
}

.queue-card span,
.queue-card p,
.queue-card small {
  color: var(--muted);
  font-weight: 800;
}

.full-loader,
.panel-loader,
.empty-state {
  display: grid;
  place-items: center;
  gap: 14px;
  min-height: 100vh;
  color: var(--teal-dark);
  font-weight: 900;
}

```

```

.panel-loader,
.empty-state {
  min-height: 280px;
  border: 1px solid var(--border);
  border-radius: 24px;
  background: white;
}

.spin {
  animation: spin 0.9s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

.metric-card,
.appointment-card,
.queue-card,
.unit-card,
.map-unit-card,
.admin-row,
.action-panel,
.form-grid-panel,
.admin-panel,
.network-card,
.announcement-form,
.mini-grid div,
.pulse-line i,
.network-bar,
.primary-action,
.admin-tool-button,
.sidebar a,
.section-header span {
  transition:
    transform 180ms ease,
    border-color 180ms ease,
    box-shadow 180ms ease,
    background-color 180ms ease,
    filter 180ms ease;
}

@media (hover: hover) and (pointer: fine) {
  .metric-card:hover,
  .appointment-card:hover,
  .queue-card:hover,
  .unit-card:hover,
  .map-unit-card:hover,
  .admin-row:hover,
  .action-panel:hover,
  .form-grid-panel:hover,
  .network-card:hover,
  .announcement-form:hover,
  .mini-grid div:hover {
    z-index: 2;
    border-color: rgba(15, 139, 141, 0.38);
    box-shadow: 0 18px 40px rgba(31, 41, 55, 0.13);
    transform: translateY(-4px);
  }

  .admin-panel:hover {
    border-color: rgba(15, 139, 141, 0.28);
    box-shadow: 0 16px 38px rgba(31, 41, 55, 0.11);
  }

  .metric-card:hover > span,
  .appointment-card:hover .icon-tile,
  .section-header:hover span {
    transform: scale(1.08) rotate(-3deg);
  }

  .pulse-line i:hover,
  .network-bar:hover {
    filter: saturate(1.22) brightness(1.04);
    transform: scaleY(1.08) translateY(-2px);
  }

  .primary-action:hover,
  .admin-tool-button:hover {
    box-shadow: 0 12px 24px rgba(15, 139, 141, 0.2);
    transform: translateY(-2px);
  }

  .sidebar a:hover svg {
    transform: scale(1.12);
  }
}

.primary-action:active,

```

```

.admin-tool-button:active {
  transform: translateY(1px);
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    transition-duration: 0.01ms !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
  }
}

@media (max-width: 1180px) {
  .metrics-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .content-grid,
  .admin-grid,
  .admin-overview-grid,
  .unit-grid,
  .map-layout {
    grid-template-columns: 1fr;
  }

  .map-panel,
  .leaflet-map {
    min-height: 540px;
  }

  .queue-board {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

@media (max-width: 900px) {
  .auth-shell {
    grid-template-columns: 1fr;
  }

  .auth-visual {
    min-height: auto;
    padding-bottom: 20px;
  }

  .auth-panel {
    padding-top: 0;
  }

  .app-shell {
    grid-template-columns: 1fr;
  }

  .sidebar {
    position: static;
    height: auto;
    padding: 16px;
    border-right: 0;
    border-bottom: 1px solid var(--border);
  }

  .sidebar nav {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }

  .sidebar a {
    justify-content: center;
    padding: 0 10px;
  }

  .sidebar a span,
  .brand-lockup.small span {
    display: none;
  }

  .workspace {
    padding: 16px;
  }

  .topbar {
    flex-direction: column;
  }

  .user-pill {
    width: 100%;
    justify-content: space-between;
  }

  .user-pill strong {

```

```

    flex: 1;
  }

  .admin-toolbar {
    grid-template-columns: 1fr 1fr;
  }

  .admin-search {
    grid-column: 1 / -1;
  }
}

@media (max-width: 620px) {
  .auth-visual {
    gap: 24px;
    padding: 22px;
  }

  .auth-copy h1 {
    font-size: 2.25rem;
  }

  .auth-panel {
    padding: 0 14px 18px;
  }

  .auth-card {
    padding: 18px;
    border-radius: 22px;
  }

  .mini-grid,
  .metrics-grid,
  .network-summary {
    grid-template-columns: 1fr;
  }

  .live-board {
    padding: 18px;
    border-radius: 22px;
  }

  .card-title-line {
    flex-direction: column;
    align-items: stretch;
  }

  .topbar,
  .action-panel,
  .admin-panel,
  .units-band {
    width: 100%;
    min-width: 0;
    padding: 16px;
    border-radius: 20px;
  }

  .photo-uploader {
    grid-template-columns: auto minmax(0, 1fr);
  }

  .photo-uploader .upload-button {
    grid-column: 1 / -1;
  }

  .admin-row {
    grid-template-columns: 1fr;
  }

  .admin-toolbar {
    grid-template-columns: 1fr;
  }

  .admin-search {
    grid-column: auto;
  }

  .admin-tool-button {
    width: 100%;
  }

  .form-grid-panel,
  .queue-board {
    grid-template-columns: 1fr;
  }

  .map-list,
  .map-panel,
  .form-grid-panel {
    border-radius: 20px;
  }
}

```



```
.appointment-card {  
  grid-template-columns: 1fr;  
}  
  
.sidebar nav {  
  grid-template-columns: 1fr;  
}  
}
```

4. Backend Node.js (server/)

API Express, banco SQLite, autenticação e regras de negocio.

Arquivo: server/db.js

Camada de persistência SQLite (better-sqlite3). Cria tabelas, seeds iniciais (usuários demo, unidades, agendamentos), publicUser e logAudit.

Total: 539 linhas | Linguagem: .js

```
// Linhas 1-294
import fs from 'node:fs';
import os from 'node:os';
import path from 'node:path';
import crypto from 'node:crypto';
import bcrypt from 'bcryptjs';
import Database from 'better-sqlite3';

const defaultDataDir = process.env.VERCEL
  ? path.join(os.tmpdir(), 'saudeconnect-data')
  : path.resolve(process.cwd(), 'data');

export const dataDir = process.env.DATA_DIR || defaultDataDir;
fs.mkdirSync(dataDir, { recursive: true });

export const db = new Database(path.join(dataDir, 'saudeconnect.sqlite'));
db.pragma('journal_mode = WAL');
db.pragma('foreign_keys = ON');

const now = () => new Date().toISOString();

export function publicUser(user) {
  if (!user) return null;
  return {
    id: user.id,
    name: user.name,
    email: user.email,
    role: user.role,
    avatar: user.avatar,
    provider: user.provider,
    createdAt: user.created_at,
    lastLogin: user.last_login,
  };
}

export function initDb() {
  db.exec(`
    CREATE TABLE IF NOT EXISTS users (
      id TEXT PRIMARY KEY,
      name TEXT NOT NULL,
      email TEXT NOT NULL UNIQUE,
      password_hash TEXT,
      role TEXT NOT NULL CHECK (role IN ('user', 'admin')) DEFAULT 'user',
      avatar TEXT,
      provider TEXT NOT NULL DEFAULT 'local',
      google_id TEXT UNIQUE,
      created_at TEXT NOT NULL,
      last_login TEXT
    );

    CREATE TABLE IF NOT EXISTS auth_sessions (
      id TEXT PRIMARY KEY,
      user_id TEXT NOT NULL,
      token_id TEXT NOT NULL UNIQUE,
      created_at TEXT NOT NULL,
      expires_at TEXT NOT NULL,
      revoked_at TEXT,
      ip_address TEXT,
      user_agent TEXT,
      FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    );

    CREATE INDEX IF NOT EXISTS idx_auth_sessions_user_id ON auth_sessions(user_id);
    CREATE INDEX IF NOT EXISTS idx_auth_sessions_token_id ON auth_sessions(token_id);

    CREATE TABLE IF NOT EXISTS units (
      id TEXT PRIMARY KEY,
      name TEXT NOT NULL,
      type TEXT NOT NULL,
      city TEXT NOT NULL,
      district TEXT NOT NULL,
      address TEXT NOT NULL,
      phone TEXT NOT NULL,
      status TEXT NOT NULL,
      services TEXT NOT NULL,
      created_at TEXT NOT NULL
    );

    CREATE TABLE IF NOT EXISTS appointments (
      id TEXT PRIMARY KEY,
      user_id TEXT NOT NULL,
      unit_id TEXT NOT NULL,
      specialty TEXT NOT NULL,
```

```

    professional TEXT,
    scheduled_at TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('pending', 'confirmed', 'completed', 'cancelled')) DEFAULT 'pending',
    reason TEXT NOT NULL,
    notes TEXT,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (unit_id) REFERENCES units(id)
);

CREATE TABLE IF NOT EXISTS exams (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    title TEXT NOT NULL,
    unit TEXT NOT NULL,
    requested_at TEXT NOT NULL,
    status TEXT NOT NULL,
    result_url TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS records (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    category TEXT NOT NULL,
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS patient_profiles (
    user_id TEXT PRIMARY KEY,
    cpf TEXT,
    birth_date TEXT,
    phone TEXT,
    sus_card TEXT,
    address TEXT,
    emergency_contact TEXT,
    updated_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS triage_cases (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    symptoms TEXT NOT NULL,
    risk_level TEXT NOT NULL CHECK (risk_level IN ('low', 'medium', 'high', 'critical')) DEFAULT 'medium',
    status TEXT NOT NULL CHECK (status IN ('waiting', 'in_service', 'resolved')) DEFAULT 'waiting',
    recommendation TEXT NOT NULL,
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS queue_entries (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    unit_id TEXT NOT NULL,
    service TEXT NOT NULL,
    position INTEGER NOT NULL,
    estimated_minutes INTEGER NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('waiting', 'called', 'done', 'cancelled')) DEFAULT 'waiting',
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (unit_id) REFERENCES units(id)
);

CREATE TABLE IF NOT EXISTS support_tickets (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL,
    subject TEXT NOT NULL,
    message TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('open', 'in_review', 'resolved')) DEFAULT 'open',
    priority TEXT NOT NULL CHECK (priority IN ('low', 'medium', 'high')) DEFAULT 'medium',
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS announcements (
    id TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    audience TEXT NOT NULL CHECK (audience IN ('all', 'users', 'admins')) DEFAULT 'all',
    published_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS integrations (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('online', 'degraded', 'offline')) DEFAULT 'online',
    last_sync TEXT NOT NULL,

```

```

    latency_ms INTEGER NOT NULL DEFAULT 0
  );

  CREATE TABLE IF NOT EXISTS audit_logs (
    id TEXT PRIMARY KEY,
    actor_id TEXT,
    action TEXT NOT NULL,
    entity TEXT NOT NULL,
    entity_id TEXT,
    created_at TEXT NOT NULL,
    FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE SET NULL
  );
`);

ensureColumn('units', 'distance_km', 'REAL NOT NULL DEFAULT 0');
ensureColumn('units', 'hours', "TEXT NOT NULL DEFAULT 'Seg-Sex: 7h às 17h'");
ensureColumn('units', 'lat', 'REAL NOT NULL DEFAULT -23.5505');
ensureColumn('units', 'lng', 'REAL NOT NULL DEFAULT -46.6333');

seedDb();
}

function ensureColumn(table, column, definition) {
  const columns = db.prepare(`PRAGMA table_info(${table})`).all().map((item) => item.name);
  if (!columns.includes(column)) {
    db.exec(`ALTER TABLE ${table} ADD COLUMN ${column} ${definition}`);
  }
}

function seedDb() {
  const seedTime = now();
  upsertUser({
    id: 'usr_admin',
    name: 'Administradora SaúdeConnect',
    email: 'admin@saudeconnect.com',
    password_hash: bcrypt.hashSync('Admin@12345', 12),
    role: 'admin',
    avatar: 'AS',
    created_at: seedTime,
    last_login: seedTime,
  });

  upsertUser({
    id: 'usr_paciente',
    name: 'Marina Costa',
    email: 'paciente@saudeconnect.com',
    password_hash: bcrypt.hashSync('Paciente@12345', 12),
    role: 'user',
    avatar: 'MC',
    created_at: seedTime,
    last_login: seedTime,
  });

  [
    {
      id: 'unit_ubs_centro',
      name: 'UBS Centro',
      type: 'Unidade Básica de Saúde',
      city: 'São Paulo',
      district: 'Centro',
      address: 'Rua da Consolação, 123 - Centro',
      phone: '(11) 3333-1111',
      status: 'Aberto',
      distance_km: 0.5,
      hours: 'Seg-Sex: 7h às 17h',
      lat: -23.5489,
      lng: -46.6388,
      services: ['Clínica geral', 'Vacinação', 'Enfermagem', 'Farmácia'],
    },
    {
      id: 'unit_ama_leste',
      name: 'AMA Leste Integrada',
      type: 'Assistência Médica Ambulatorial',
      city: 'São Paulo',
      district: 'Tatuapé',
      address: 'Rua Serra de Bragança, 890 - Tatuapé',
      phone: '(11) 3333-2222',
      status: 'Aberto',
      distance_km: 2.1,
      hours: 'Todos os dias: 7h às 19h',
      lat: -23.5401,
      lng: -46.5767,
      services: ['Pediatria', 'Ortopedia', 'Curativos', 'Triagem'],
    },
    {
      id: 'unit_clinica_norte',
      name: 'Clínica Norte Digital',
      type: 'Clínica parceira',
      city: 'São Paulo',
      district: 'Santana',
      address: 'Rua Voluntários da Pátria, 1440 - Santana',
    }
  ]
}

```

```

    phone: '(11) 3333-5555',
    status: 'Alta demanda',
    distance_km: 2.8,
    hours: 'Seg-Sex: 8h às 20h',
    lat: -23.5024,
    lng: -46.6247,
    services: ['Cardiologia', 'Pediatria', 'Teleconsulta'],
  },
  {
    id: 'unit_lab_viva',
    name: 'Laboratório Viva Saúde',
    type: 'Laboratório',
    city: 'São Paulo',
    district: 'Bela Vista',
    address: 'Av. Paulista, 980 - Bela Vista',
    phone: '(11) 3333-3333',
    status: 'Coleta até 16h',
    distance_km: 1.3,
    hours: 'Seg-Sáb: 6h às 16h',
    lat: -23.5652,
    lng: -46.6524,
    services: ['Hemograma', 'Imagem', 'Resultados digitais'],
  },
  {
    id: 'unit_caps_oeste',
    name: 'CAPS Oeste',
    type: 'Atenção Psicossocial',
    city: 'São Paulo',
    district: 'Pinheiros',
  },
// Linhas 295-540
  address: 'Rua Cardeal Arcoverde, 440 - Pinheiros',
  phone: '(11) 3333-4444',
  status: 'Plantão reduzido',
  distance_km: 3.4,
  hours: 'Seg-Sex: 8h às 18h',
  lat: -23.5617,
  lng: -46.6816,
  services: ['Psicologia', 'Psiquiatria', 'Acolhimento'],
},
].forEach(upsertUnit);

upsertProfile({
  user_id: 'usr_paciente',
  cpf: '123.456.789-00',
  birth_date: '1991-04-18',
  phone: '(11) 98888-2026',
  sus_card: '898 0011 2222 3333',
  address: 'Rua Augusta, 500 - Consolação',
  emergency_contact: 'João Costa - (11) 97777-1010',
  updated_at: seedTime,
});

seedOnce('appointments', 'apt_001', () =>
  db.prepare(`
    INSERT INTO appointments (id, user_id, unit_id, specialty, professional, scheduled_at, status, reason, notes, c...
    VALUES (@id, @user_id, @unit_id, @specialty, @professional, @scheduled_at, @status, @reason, @notes, @created_at)
  `).run({
    id: 'apt_001',
    user_id: 'usr_paciente',
    unit_id: 'unit_ubs_centro',
    specialty: 'Clínica geral',
    professional: 'Dra. Ana Ribeiro',
    scheduled_at: new Date(Date.now() + 86400000 * 2).toISOString(),
    status: 'confirmed',
    reason: 'Acompanhamento de pressão arterial',
    notes: 'Chegar 15 minutos antes.',
    created_at: seedTime,
  })),
);

seedOnce('appointments', 'apt_002', () =>
  db.prepare(`
    INSERT INTO appointments (id, user_id, unit_id, specialty, professional, scheduled_at, status, reason, notes, c...
    VALUES (@id, @user_id, @unit_id, @specialty, @professional, @scheduled_at, @status, @reason, @notes, @created_at)
  `).run({
    id: 'apt_002',
    user_id: 'usr_paciente',
    unit_id: 'unit_ama_leste',
    specialty: 'Cardiologia',
    professional: 'Dr. Felipe Torres',
    scheduled_at: new Date(Date.now() + 86400000 * 9).toISOString(),
    status: 'pending',
    reason: 'Avaliar exame de rotina',
    notes: '',
    created_at: seedTime,
  })),
);

[
  ['exam_001', 'Hemograma completo', 'Laboratório Viva Saúde', 'Resultado disponível', '#'],

```

```

['exam_002', 'Eletrocardiograma', 'AMA Leste Integrada', 'Em análise', null],
].forEach(([id, title, unit, status, result_url], index) =>
  seedOnce('exams', id, () =>
    db.prepare(`
      INSERT INTO exams (id, user_id, title, unit, requested_at, status, result_url)
      VALUES (@id, @user_id, @title, @unit, @requested_at, @status, @result_url)
    `).run({
      id,
      user_id: 'usr_paciente',
      title,
      unit,
      requested_at: new Date(Date.now() - 86400000 * (index + 4)).toISOString(),
      status,
      result_url,
    })),
);

[
  ['rec_001', 'Receita', 'Losartana 50mg', 'Tomar 1 comprimido pela manhã por 60 dias.'],
  ['rec_002', 'Histórico', 'Consulta de enfermagem', 'Orientações sobre sono, hidratação e aferição semanal.'],
  ['rec_003', 'Vacina', 'Influenza', 'Dose registrada na UBS Centro.'],
].forEach(([id, category, title, description], index) =>
  seedOnce('records', id, () =>
    db.prepare(`
      INSERT INTO records (id, user_id, category, title, description, created_at)
      VALUES (@id, @user_id, @category, @title, @description, @created_at)
    `).run({
      id,
      user_id: 'usr_paciente',
      category,
      title,
      description,
      created_at: new Date(Date.now() - 86400000 * (index + 2)).toISOString(),
    })),
);

seedOnce('triage_cases', 'tri_001', () =>
  db.prepare(`
    INSERT INTO triage_cases (id, user_id, symptoms, risk_level, status, recommendation, created_at)
    VALUES (@id, @user_id, @symptoms, @risk_level, @status, @recommendation, @created_at)
  `).run({
    id: 'tri_001',
    user_id: 'usr_paciente',
    symptoms: 'Dor de cabeça persistente e pressão elevada.',
    risk_level: 'medium',
    status: 'waiting',
    recommendation: 'Prioridade amarela. Comparecer à UBS Centro ou aguardar chamada da equipe.',
    created_at: seedTime,
  })),
);

seedOnce('queue_entries', 'queue_001', () =>
  db.prepare(`
    INSERT INTO queue_entries (id, user_id, unit_id, service, position, estimated_minutes, status, created_at)
    VALUES (@id, @user_id, @unit_id, @service, @position, @estimated_minutes, @status, @created_at)
  `).run({
    id: 'queue_001',
    user_id: 'usr_paciente',
    unit_id: 'unit_ubs_centro',
    service: 'Clínica geral',
    position: 3,
    estimated_minutes: 28,
    status: 'waiting',
    created_at: seedTime,
  })),
);

seedOnce('support_tickets', 'tic_001', () =>
  db.prepare(`
    INSERT INTO support_tickets (id, user_id, subject, message, status, priority, created_at)
    VALUES (@id, @user_id, @subject, @message, @status, @priority, @created_at)
  `).run({
    id: 'tic_001',
    user_id: 'usr_paciente',
    subject: 'Atualizar telefone cadastrado',
    message: 'Preciso alterar meu telefone de contato para receber lembretes.',
    status: 'in_review',
    priority: 'medium',
    created_at: seedTime,
  })),
);

[
  ['ann_001', 'Campanha de vacinação aberta', 'Novos horários foram liberados nas unidades integradas.', 'all'],
  ['ann_002', 'Fila digital atualizada', 'Administradores já podem acompanhar prioridades em tempo real.', 'admins'],
].forEach(([id, title, body, audience]) =>
  seedOnce('announcements', id, () =>
    db.prepare(`
      INSERT INTO announcements (id, title, body, audience, published_at)
    `).run({
      id,
      title,
      body,
      audience,
      published_at: seedTime,
    })),
);

```

```

        VALUES (@id, @title, @body, @audience, @published_at)
    `).run({ id, title, body, audience, published_at: seedTime }},
),
);

[
  ['int_001', 'Cadastro Nacional de Usuários', 'online', 128],
  ['int_002', 'Agenda municipal', 'online', 214],
  ['int_003', 'Laboratórios parceiros', 'degraded', 680],
  ['int_004', 'Notificações SMS', 'online', 190],
].forEach(([id, name, status, latency_ms]) =>
  seedOnce('integrations', id, () =>
    db.prepare(`
      INSERT INTO integrations (id, name, status, last_sync, latency_ms)
      VALUES (@id, @name, @status, @last_sync, @latency_ms)
    `).run({ id, name, status, last_sync: seedTime, latency_ms }},
  ),
);
}

function seedOnce(table, id, insert) {
  const exists = db.prepare(`SELECT id FROM ${table} WHERE id = ?`).get(id);
  if (!exists) insert();
}

function upsertUser(user) {
  const exists = db.prepare('SELECT id FROM users WHERE id = ? OR email = ?').get(user.id, user.email);
  if (exists) {
    db.prepare(`
      UPDATE users
      SET name = @name, role = @role, avatar = @avatar
      WHERE id = @id OR email = @email
    `).run(user);
    return;
  }

  db.prepare(`
    INSERT INTO users (id, name, email, password_hash, role, avatar, provider, created_at, last_login)
    VALUES (@id, @name, @email, @password_hash, @role, @avatar, 'local', @created_at, @last_login)
  `).run(user);
}

function upsertUnit(unit) {
  db.prepare(`
    INSERT INTO units (id, name, type, city, district, address, phone, status, services, distance_km, hours, lat, lng...
    VALUES (@id, @name, @type, @city, @district, @address, @phone, @status, @services, @distance_km, @hours, @lat, @l...
    ON CONFLICT(id) DO UPDATE SET
      name = excluded.name,
      type = excluded.type,
      city = excluded.city,
      district = excluded.district,
      address = excluded.address,
      phone = excluded.phone,
      status = excluded.status,
      services = excluded.services,
      distance_km = excluded.distance_km,
      hours = excluded.hours,
      lat = excluded.lat,
      lng = excluded.lng
  `).run({
    ...unit,
    services: JSON.stringify(unit.services),
    created_at: now(),
  });
}

function upsertProfile(profile) {
  db.prepare(`
    INSERT INTO patient_profiles (user_id, cpf, birth_date, phone, sus_card, address, emergency_contact, updated_at)
    VALUES (@user_id, @cpf, @birth_date, @phone, @sus_card, @address, @emergency_contact, @updated_at)
    ON CONFLICT(user_id) DO UPDATE SET
      cpf = excluded.cpf,
      birth_date = excluded.birth_date,
      phone = excluded.phone,
      sus_card = excluded.sus_card,
      address = excluded.address,
      emergency_contact = excluded.emergency_contact,
      updated_at = excluded.updated_at
  `).run(profile);
}

export function logAudit(actorId, action, entity, entityId) {
  db.prepare(`
    INSERT INTO audit_logs (id, actor_id, action, entity, entity_id, created_at)
    VALUES (@id, @actor_id, @action, @entity, @entity_id, @created_at)
  `).run({
    id: crypto.randomUUID(),
    actor_id: actorId,
    action,
    entity,
    entity_id: entityId,
  });
}

```

```
        created_at: now(),  
    });  
}
```


Arquivo: server/auth.js

Autenticação JWT: signToken, authRequired, adminRequired, revogacao de sessao. Adaptacao para Vercel serverless sem persistência de sessoes.

Total: 118 linhas | Linguagem: .js

```
import crypto from 'node:crypto';
import jwt from 'jsonwebtoken';
import { db, publicUser } from './db.js';

const JWT_SECRET = process.env.JWT_SECRET || 'dev-secret-change-me-before-production';
const TOKEN_TTL = process.env.JWT_TTL || '7d';
const isVercel = Boolean(process.env.VERCEL);

export function signToken(user, req) {
  const tokenId = crypto.randomUUID();
  const token = jwt.sign(
    {
      sub: user.id,
      role: user.role,
      jti: tokenId,
      name: user.name,
      email: user.email,
      avatar: user.avatar,
      provider: user.provider || 'local',
      createdAt: user.created_at,
      lastLogin: user.last_login,
    },
    JWT_SECRET,
    { expiresIn: TOKEN_TTL },
  );
  const decoded = jwt.decode(token);

  if (!isVercel) {
    db.prepare(`
      INSERT INTO auth_sessions (id, user_id, token_id, created_at, expires_at, ip_address, user_agent)
      VALUES (@id, @user_id, @token_id, @created_at, @expires_at, @ip_address, @user_agent)
    `).run({
      id: crypto.randomUUID(),
      user_id: user.id,
      token_id: tokenId,
      created_at: new Date().toISOString(),
      expires_at: new Date(Number(decoded.exp) * 1000).toISOString(),
      ip_address: req?.ip || null,
      user_agent: req?.get?('(user-agent)') || null,
    });
  }

  return token;
}

export function authRequired(req, res, next) {
  const header = req.get('authorization') || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;

  if (!token) {
    return res.status(401).json({ message: 'Sessao necessaria.' });
  }

  try {
    const payload = jwt.verify(token, JWT_SECRET);
    if (!payload.jti) {
      return res.status(401).json({ message: 'Sessao expirada ou invalida.' });
    }
  }

  const session = isVercel
    ? null
    : db.prepare('SELECT * FROM auth_sessions WHERE token_id = ? AND user_id = ?').get(payload.jti, payload.sub);

  if (!isVercel && (!session || session.revoked_at || Date.parse(session.expires_at) <= Date.now())) {
    return res.status(401).json({ message: 'Sessao expirada ou invalida.' });
  }

  let user = db
    .prepare('SELECT * FROM users WHERE id = ?')
    .get(payload.sub);

  if (!user && isVercel && payload.email && payload.name) {
    db.prepare(`
      INSERT OR IGNORE INTO users
      (id, name, email, password_hash, role, avatar, provider, created_at, last_login)
      VALUES
      (@id, @name, @email, NULL, @role, @avatar, @provider, @created_at, @last_login)
    `).run({
      id: payload.sub,
      name: payload.name,
      email: payload.email,
      role: payload.role === 'admin' ? 'admin' : 'user',
      avatar: payload.avatar || null,
    });
  }
}
```

```

        provider: payload.provider || 'local',
        created_at: payload.createdAt || new Date().toISOString(),
        last_login: payload.lastLogin || new Date().toISOString(),
    });
    user = db.prepare('SELECT * FROM users WHERE id = ? OR email = ?').get(payload.sub, payload.email);
}

if (!user) {
    return res.status(401).json({ message: 'Usuario nao encontrado.' });
}

req.user = user;
req.publicUser = publicUser(user);
req.authSession = session || { id: null, token_id: payload.jti };
return next();
} catch {
    return res.status(401).json({ message: 'Sessao expirada ou invalida.' });
}
}

export function revokeCurrentSession(req) {
    if (!req.authSession?.id) return;
    db.prepare('UPDATE auth_sessions SET revoked_at = ? WHERE id = ? AND revoked_at IS NULL').run(
        new Date().toISOString(),
        req.authSession.id,
    );
}

export function adminRequired(req, res, next) {
    if (req.user?.role !== 'admin') {
        return res.status(403).json({ message: 'Acesso restrito a administradores.' });
    }
    return next();
}
}

```

Arquivo: server/routes.js

Rotas REST da API: auth (local/Google), dashboard, perfil, prontuários, triagem, fila, agendamentos, suporte e endpoints administrativos com Zod.

Total: 839 linhas | Linguagem: .js

```
// Linhas 1-264
import crypto from 'node:crypto';
import fs from 'node:fs';
import path from 'node:path';
import bcrypt from 'bcryptjs';
import express from 'express';
import passport from 'passport';
import { Strategy as GoogleStrategy } from 'passport-google-oauth20';
import { z } from 'zod';
import { authRequired, adminRequired, revokeCurrentSession, signToken } from './auth.js';
import { dataDir, db, logAudit, publicUser } from './db.js';

const router = express.Router();
const now = () => new Date().toISOString();

const googleEnabled =
  Boolean(process.env.GOOGLE_CLIENT_ID) &&
  Boolean(process.env.GOOGLE_CLIENT_SECRET) &&
  Boolean(process.env.GOOGLE_CALLBACK_URL);

if (googleEnabled) {
  passport.use(
    new GoogleStrategy(
      {
        clientID: process.env.GOOGLE_CLIENT_ID,
        clientSecret: process.env.GOOGLE_CLIENT_SECRET,
        callbackURL: process.env.GOOGLE_CALLBACK_URL,
      },
      (_accessToken, _refreshToken, profile, done) => {
        try {
          const email = profile.emails?.[0]?.value?.toLowerCase();
          if (!email) return done(null, false);

          const existing = db
            .prepare('SELECT * FROM users WHERE google_id = ? OR email = ?')
            .get(profile.id, email);

          if (existing) {
            db.prepare(
              'UPDATE users SET google_id = ?, provider = ?, avatar = COALESCE(avatar, ?), last_login = ? WHERE id = ...'
            ).run(profile.id, 'google', initials(profile.displayName), now(), existing.id);

            const refreshed = db.prepare('SELECT * FROM users WHERE id = ?').get(existing.id);
            logAudit(existing.id, 'logged_in_google', 'users', existing.id);
            return done(null, { ...refreshed, is_new_user: false });
          }

          const user = {
            id: crypto.randomUUID(),
            name: profile.displayName || email.split('@')[0],
            email,
            avatar: initials(profile.displayName || email),
            created_at: now(),
          };

          db.prepare(`
            INSERT INTO users (id, name, email, password_hash, role, avatar, provider, google_id, created_at, last_lo...
            VALUES (@id, @name, @email, NULL, 'user', @avatar, 'google', @google_id, @created_at, @last_login)
          `).run({ ...user, google_id: profile.id, last_login: user.created_at });

          const created = db.prepare('SELECT * FROM users WHERE id = ?').get(user.id);
          logAudit(user.id, 'registered_google', 'users', user.id);
          return done(null, { ...created, is_new_user: true });
        } catch (error) {
          return done(error);
        }
      }
    ),
  );
}

const passwordSchema = z
  .string()
  .min(10, 'A senha precisa ter pelo menos 10 caracteres.')
  .max(128, 'A senha deve ter no maximo 128 caracteres.')
  .regex(/[a-z]/, 'A senha precisa ter uma letra minuscula.')
  .regex(/[A-Z]/, 'A senha precisa ter uma letra maiuscula.')
  .regex(/[0-9]/, 'A senha precisa ter um numero.')
  .regex(/^[^A-Za-z0-9]/, 'A senha precisa ter um caractere especial.');
```

```
const registerSchema = z.object({
  name: z.string().min(3, 'Informe seu nome completo.'),
  email: z.string().email('Informe um e-mail valido.'),
```

```

    password: passwordSchema,
  });

  const loginSchema = z.object({
    email: z.string().email(),
    password: z.string().min(1),
  });

  const appointmentSchema = z.object({
    unitId: z.string().min(1),
    specialty: z.string().min(2),
    scheduledAt: z.string().min(6),
    reason: z.string().min(6),
  });

  const ticketSchema = z.object({
    subject: z.string().min(4),
    message: z.string().min(8),
    priority: z.enum(['low', 'medium', 'high']).default('medium'),
  });

  const profileSchema = z.object({
    cpf: z.string().min(3).optional().default(''),
    birthDate: z.string().optional().default(''),
    phone: z.string().min(3).optional().default(''),
    susCard: z.string().optional().default(''),
    address: z.string().optional().default(''),
    emergencyContact: z.string().optional().default(''),
  });

  const avatarSchema = z.object({
    imageData: z.string().min(1),
  });

  const recordSchema = z.object({
    category: z.string().min(2),
    title: z.string().min(3),
    description: z.string().min(6),
  });

  const triageSchema = z.object({
    symptoms: z.string().min(8),
    riskLevel: z.enum(['low', 'medium', 'high', 'critical']).default('medium'),
  });

  const queueSchema = z.object({
    unitId: z.string().min(1),
    service: z.string().min(2),
  });

  const statusSchema = z.object({
    status: z.string().min(2),
  });

  const roleSchema = z.object({
    role: z.enum(['user', 'admin']),
  });

  const announcementSchema = z.object({
    title: z.string().min(4, 'Informe um titulo com pelo menos 4 caracteres.').max(100),
    body: z.string().min(10, 'Informe uma mensagem com pelo menos 10 caracteres.').max(500),
    audience: z.enum(['all', 'users', 'admins']).default('all'),
  });

  router.get('/health', (_req, res) => {
    res.json({ ok: true, service: 'SaudeConnect API', time: now() });
  });

  router.get('/bootstrap', (_req, res) => {
    res.json({
      googleEnabled,
    });
  });

  router.post('/auth/register', (req, res, next) => {
    try {
      const input = registerSchema.parse(req.body);
      const email = input.email.toLowerCase();
      const exists = db.prepare('SELECT id FROM users WHERE email = ?').get(email);

      if (exists) {
        return res.status(409).json({ message: 'E-mail ja cadastrado.' });
      }

      const user = {
        id: crypto.randomUUID(),
        name: input.name,
        email,
        password_hash: bcrypt.hashSync(input.password, 12),
        role: 'user',
        avatar: initials(input.name),
      };
    }
  });

```

```

        created_at: now(),
        last_login: now(),
    });

    db.prepare(`
        INSERT INTO users (id, name, email, password_hash, role, avatar, provider, created_at, last_login)
        VALUES (@id, @name, @email, @password_hash, @role, @avatar, 'local', @created_at, @last_login)
    `).run(user);

    const token = signToken(user, req);
    logAudit(user.id, 'registered', 'users', user.id);
    return res.status(201).json({ user: publicUser(user), token });
} catch (error) {
    return next(error);
}
});

router.post('/auth/login', (req, res, next) => {
    try {
        const input = loginSchema.parse(req.body);
        const user = db.prepare('SELECT * FROM users WHERE email = ?').get(input.email.toLowerCase());

        if (!user || !user.password_hash || !bcrypt.compareSync(input.password, user.password_hash)) {
            return res.status(401).json({ message: 'Credenciais invalidas.' });
        }

        db.prepare('UPDATE users SET last_login = ? WHERE id = ?').run(now(), user.id);
        const refreshed = db.prepare('SELECT * FROM users WHERE id = ?').get(user.id);
        logAudit(user.id, 'logged_in', 'users', user.id);
        return res.json({ user: publicUser(refreshed), token: signToken(refreshed, req) });
    } catch (error) {
        return next(error);
    }
});

router.get('/auth/google', (req, res, next) => {
    if (!googleEnabled) {
        return res.status(503).json({
            message: 'Login Google indisponivel. Configure GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET e GOOGLE_CALLBACK_URL.'
        });
    }

    return passport.authenticate('google', { scope: ['profile', 'email'], session: false })(req, res, next);
});

router.get(
    '/auth/google/callback',
    passport.authenticate('google', { session: false, failureRedirect: '/login?oauth=failed' }),
    (req, res) => {
        const token = signToken(req.user, req);
        const forwardedProto = req.get('x-forwarded-proto') || req.protocol;
        const forwardedHost = req.get('x-forwarded-host') || req.get('host');
        const clientUrl = process.env.CLIENT_URL || `${forwardedProto}://${forwardedHost}`;
        const params = new URLSearchParams({ token });
        if (req.user?.is_new_user) params.set('newUser', '1');
        res.redirect(`${clientUrl}/auth/callback?${params.toString()}`);
    },
);

router.get('/auth/me', authRequired, (req, res) => {
    res.json({ user: req.publicUser });
});

router.post('/auth/avatar', authRequired, (req, res, next) => {
    try {
        const { imageData } = avatarSchema.parse(req.body);
        const match = imageData.match(/^data:(image\/(?:png|jpeg|webp));base64,([A-Za-z0-9+\/=]*)$/);

        if (!match) {
            return res.status(400).json({ message: 'Envie uma imagem PNG, JPG ou WebP.' });
        }

        const mimeType = match[1];
        const extension = mimeType.includes('png') ? 'png' : mimeType.includes('webp') ? 'webp' : 'jpg';
        const buffer = Buffer.from(match[2], 'base64');
        const maxBytes = 2 * 1024 * 1024;

        if (buffer.length > maxBytes) {
            return res.status(400).json({ message: 'A foto deve ter no maximo 2 MB.' });
        }

        if (!isValidImageBuffer(buffer, mimeType)) {
            return res.status(400).json({ message: 'O arquivo enviado nao parece ser uma imagem valida.' });
        }

        const uploadsDir = path.join(dataDir, 'uploads', 'avatars');
        fs.mkdirSync(uploadsDir, { recursive: true });

        const previousAvatar = req.user.avatar;
        const fileName = `${req.user.id}-${crypto.randomUUID()}.${extension}`;
    }
});

```

```
// Linhas 265-529
const filePath = path.join(uploadsDir, fileName);
fs.writeFileSync(filePath, buffer);

const avatarUrl = process.env.VERCEL
  ? `/api/uploads/avatars/${fileName}`
  : `/uploads/avatars/${fileName}`;
db.prepare('UPDATE users SET avatar = ? WHERE id = ?').run(avatarUrl, req.user.id);

if (/^\/(?:api\/)?uploads\/avatars\/\/.test(previousAvatar || '')) {
  const previousFile = path.join(
    dataDir,
    previousAvatar.replace(/^\/(?:api\/)?uploads\/\/, 'uploads/'),
  );
  fs.rmSync(previousFile, { force: true });
}

logAudit(req.user.id, 'updated_avatar', 'users', req.user.id);
const user = db.prepare('SELECT * FROM users WHERE id = ?').get(req.user.id);
return res.json({ user: publicUser(user) });
} catch (error) {
  return next(error);
}
});

router.post('/auth/logout', authRequired, (req, res) => {
  revokeCurrentSession(req);
  logAudit(req.user.id, 'logged_out', 'auth_sessions', req.authSession.id);
  res.json({ ok: true });
});

router.get('/units', authRequired, (_req, res) => {
  const units = listUnits();
  res.json({ units });
});

router.get('/dashboard', authRequired, (req, res) => {
  const appointments = appointmentsForUser(req.user.id);
  const exams = db.prepare('SELECT * FROM exams WHERE user_id = ? ORDER BY requested_at DESC').all(req.user.id);
  const records = db.prepare('SELECT * FROM records WHERE user_id = ? ORDER BY created_at DESC').all(req.user.id);
  const profile = profileForUser(req.user.id);
  const triage = triageForUser(req.user.id);
  const queue = queueForUser(req.user.id);
  const tickets = db
    .prepare('SELECT * FROM support_tickets WHERE user_id = ? ORDER BY created_at DESC')
    .all(req.user.id);
  const units = listUnits();
  const announcements = db
    .prepare('SELECT * FROM announcements WHERE audience IN ('all', 'users') ORDER BY published_at DESC LIMIT 4')
    .all();

  res.json({
    user: req.publicUser,
    metrics: {
      nextAppointments: appointments.filter((item) => ['pending', 'confirmed'].includes(item.status)).length,
      availableResults: exams.filter((item) => item.status.toLowerCase().includes('disponivel')).length,
      activeTickets: tickets.filter((item) => item.status !== 'resolved').length,
      connectedUnits: units.length,
      activeTriage: triage.filter((item) => item.status !== 'resolved').length,
      queuePosition: queue.find((item) => item.status === 'waiting')?.position || 0,
    },
    appointments,
    exams,
    records,
    profile,
    triage,
    queue,
    tickets,
    units,
    announcements,
  });
});

router.get('/profile', authRequired, (req, res) => {
  res.json({ profile: profileForUser(req.user.id) });
});

router.put('/profile', authRequired, (req, res, next) => {
  try {
    const input = profileSchema.parse(req.body);
    db.prepare(`
      INSERT INTO patient_profiles (user_id, cpf, birth_date, phone, sus_card, address, emergency_contact, updated_at)
      VALUES (@user_id, @cpf, @birth_date, @phone, @sus_card, @address, @emergency_contact, @updated_at)
      ON CONFLICT(user_id) DO UPDATE SET
        cpf = excluded.cpf,
        birth_date = excluded.birth_date,
        phone = excluded.phone,
        sus_card = excluded.sus_card,
        address = excluded.address,
        emergency_contact = excluded.emergency_contact,
        updated_at = excluded.updated_at
    `);
  } catch (error) {
    next(error);
  }
});
```

```

    `).run({
      user_id: req.user.id,
      cpf: input.cpf,
      birth_date: input.birthDate,
      phone: input.phone,
      sus_card: input.susCard,
      address: input.address,
      emergency_contact: input.emergencyContact,
      updated_at: now(),
    });

    logAudit(req.user.id, 'updated', 'patient_profiles', req.user.id);
    return res.json({ profile: profileForUser(req.user.id) });
  } catch (error) {
    return next(error);
  }
});

router.get('/records', authRequired, (req, res) => {
  const records = db.prepare('SELECT * FROM records WHERE user_id = ? ORDER BY created_at DESC').all(req.user.id);
  res.json({ records });
});

router.post('/records', authRequired, (req, res, next) => {
  try {
    const input = recordSchema.parse(req.body);
    const record = {
      id: crypto.randomUUID(),
      user_id: req.user.id,
      category: input.category,
      title: input.title,
      description: input.description,
      created_at: now(),
    };

    db.prepare(`
      INSERT INTO records (id, user_id, category, title, description, created_at)
      VALUES (@id, @user_id, @category, @title, @description, @created_at)
    `).run(record);

    logAudit(req.user.id, 'created', 'records', record.id);
    return res.status(201).json({ record });
  } catch (error) {
    return next(error);
  }
});

router.get('/triage', authRequired, (req, res) => {
  res.json({ triage: triageForUser(req.user.id) });
});

router.post('/triage', authRequired, (req, res, next) => {
  try {
    const input = triageSchema.parse(req.body);
    const recommendation = recommendationForRisk(input.riskLevel);
    const triageCase = {
      id: crypto.randomUUID(),
      user_id: req.user.id,
      symptoms: input.symptoms,
      risk_level: input.riskLevel,
      status: input.riskLevel === 'critical' ? 'in_service' : 'waiting',
      recommendation,
      created_at: now(),
    };

    db.prepare(`
      INSERT INTO triage_cases (id, user_id, symptoms, risk_level, status, recommendation, created_at)
      VALUES (@id, @user_id, @symptoms, @risk_level, @status, @recommendation, @created_at)
    `).run(triageCase);

    logAudit(req.user.id, 'created', 'triage_cases', triageCase.id);
    return res.status(201).json({ triage: triageCase });
  } catch (error) {
    return next(error);
  }
});

router.get('/queue', authRequired, (req, res) => {
  res.json({ queue: queueForUser(req.user.id) });
});

router.post('/queue', authRequired, (req, res, next) => {
  try {
    const input = queueSchema.parse(req.body);
    const unit = db.prepare('SELECT * FROM units WHERE id = ?').get(input.unitId);
    if (!unit) return res.status(404).json({ message: 'Unidade não encontrada.' });

    const nextPosition =
      db.prepare("SELECT COALESCE(MAX(position), 0) + 1 AS next FROM queue_entries WHERE unit_id = ? AND status = 'wa...")
        .get(input.unitId).next || 1;
    const queueEntry = {

```

```

        id: crypto.randomUUID(),
        user_id: req.user.id,
        unit_id: input.unitId,
        service: input.service,
        position: nextPosition,
        estimated_minutes: nextPosition * 9 + 7,
        status: 'waiting',
        created_at: now(),
    };

    db.prepare(`
        INSERT INTO queue_entries (id, user_id, unit_id, service, position, estimated_minutes, status, created_at)
        VALUES (@id, @user_id, @unit_id, @service, @position, @estimated_minutes, @status, @created_at)
    `).run(queueEntry);

    logAudit(req.user.id, 'created', 'queue_entries', queueEntry.id);
    return res.status(201).json({ queue: queueById(queueEntry.id) });
} catch (error) {
    return next(error);
}
});

router.post('/appointments', authRequired, (req, res, next) => {
    try {
        const input = appointmentSchema.parse(req.body);
        const unit = db.prepare('SELECT * FROM units WHERE id = ?').get(input.unitId);

        if (!unit) {
            return res.status(404).json({ message: 'Unidade nao encontrada.' });
        }

        const appointment = {
            id: crypto.randomUUID(),
            user_id: req.user.id,
            unit_id: input.unitId,
            specialty: input.specialty,
            professional: 'A definir',
            scheduled_at: new Date(input.scheduledAt).toISOString(),
            status: 'pending',
            reason: input.reason,
            notes: '',
            created_at: now(),
        };

        db.prepare(`
            INSERT INTO appointments (id, user_id, unit_id, specialty, professional, scheduled_at, status, reason, notes, c...
            VALUES (@id, @user_id, @unit_id, @specialty, @professional, @scheduled_at, @status, @reason, @notes, @created_at)
        `).run(appointment);

        logAudit(req.user.id, 'created', 'appointments', appointment.id);
        return res.status(201).json({ appointment: appointmentById(appointment.id) });
    } catch (error) {
        return next(error);
    }
});

router.post('/support', authRequired, (req, res, next) => {
    try {
        const input = ticketSchema.parse(req.body);
        const ticket = {
            id: crypto.randomUUID(),
            user_id: req.user.id,
            subject: input.subject,
            message: input.message,
            status: 'open',
            priority: input.priority,
            created_at: now(),
        };

        db.prepare(`
            INSERT INTO support_tickets (id, user_id, subject, message, status, priority, created_at)
            VALUES (@id, @user_id, @subject, @message, @status, @priority, @created_at)
        `).run(ticket);

        logAudit(req.user.id, 'created', 'support_tickets', ticket.id);
        return res.status(201).json({ ticket });
    } catch (error) {
        return next(error);
    }
});

router.get('/admin/overview', authRequired, adminRequired, (_req, res) => {
    const overview = {
// Linhas 530-765
        users: count('users'),
        appointments: count('appointments'),
        openTickets: db.prepare("SELECT COUNT(*) AS total FROM support_tickets WHERE status != 'resolved'").get().total,
        integrationsOnline: db.prepare("SELECT COUNT(*) AS total FROM integrations WHERE status = 'online'").get().total,
        triageWaiting: db.prepare("SELECT COUNT(*) AS total FROM triage_cases WHERE status != 'resolved'").get().total,
        queueWaiting: db.prepare("SELECT COUNT(*) AS total FROM queue_entries WHERE status = 'waiting'").get().total,
    };

```



```

});

const appointments = db
  .prepare(`
    SELECT appointments.*, users.name AS user_name, users.email AS user_email, units.name AS unit_name
    FROM appointments
    JOIN users ON users.id = appointments.user_id
    JOIN units ON units.id = appointments.unit_id
    ORDER BY scheduled_at DESC
    LIMIT 12
  `)
  .all();
const users = db
  .prepare('SELECT id, name, email, role, avatar, provider, created_at, last_login FROM users ORDER BY created_at D...')
  .all();
const tickets = db
  .prepare(`
    SELECT support_tickets.*, users.name AS user_name, users.email AS user_email
    FROM support_tickets
    JOIN users ON users.id = support_tickets.user_id
    ORDER BY created_at DESC
  `)
  .all();
const triage = db
  .prepare(`
    SELECT triage_cases.*, users.name AS user_name, users.email AS user_email
    FROM triage_cases
    JOIN users ON users.id = triage_cases.user_id
    ORDER BY triage_cases.created_at DESC
  `)
  .all();
const queue = db
  .prepare(`
    SELECT queue_entries.*, users.name AS user_name, users.email AS user_email, units.name AS unit_name
    FROM queue_entries
    JOIN users ON users.id = queue_entries.user_id
    JOIN units ON units.id = queue_entries.unit_id
    ORDER BY queue_entries.status = 'waiting' DESC, queue_entries.position ASC
  `)
  .all();
const integrations = db.prepare('SELECT * FROM integrations ORDER BY name').all();
const announcements = db.prepare('SELECT * FROM announcements ORDER BY published_at DESC LIMIT 8').all();
const auditLogs = db
  .prepare(`
    SELECT audit_logs.*, users.name AS actor_name
    FROM audit_logs
    LEFT JOIN users ON users.id = audit_logs.actor_id
    ORDER BY audit_logs.created_at DESC
    LIMIT 10
  `)
  .all();

res.json({ overview, appointments, users, tickets, triage, queue, integrations, announcements, auditLogs });
});

router.patch('/admin/users/:id/role', authRequired, adminRequired, (req, res, next) => {
  try {
    const input = roleSchema.parse(req.body);
    if (req.params.id !== req.user.id && input.role !== 'admin') {
      return res.status(400).json({ message: 'Voce nao pode remover sua propria permissao de administrador.' });
    }

    const result = db.prepare('UPDATE users SET role = ? WHERE id = ?').run(input.role, req.params.id);
    if (result.changes === 0) return res.status(404).json({ message: 'Usuario nao encontrado.' });

    logAudit(req.user.id, 'updated_role', 'users', req.params.id);
    return res.json({ user: publicUser(db.prepare('SELECT * FROM users WHERE id = ?').get(req.params.id)) });
  } catch (error) {
    return next(error);
  }
});

router.post('/admin/announcements', authRequired, adminRequired, (req, res, next) => {
  try {
    const input = announcementSchema.parse(req.body);
    const announcement = {
      id: crypto.randomUUID(),
      title: input.title.trim(),
      body: input.body.trim(),
      audience: input.audience,
      published_at: now(),
    };

    db.prepare(`
      INSERT INTO announcements (id, title, body, audience, published_at)
      VALUES (@id, @title, @body, @audience, @published_at)
    `).run(announcement);
    logAudit(req.user.id, 'published', 'announcements', announcement.id);
    return res.status(201).json({ announcement });
  } catch (error) {
    return next(error);
  }
});

```

```

    }
  });

  router.patch('/admin/appointments/:id', authRequired, adminRequired, (req, res, next) => {
    try {
      const input = statusSchema.parse(req.body);
      if (!['pending', 'confirmed', 'completed', 'cancelled'].includes(input.status)) {
        return res.status(400).json({ message: 'Status de agendamento invalido.' });
      }

      const result = db.prepare('UPDATE appointments SET status = ? WHERE id = ?').run(input.status, req.params.id);
      if (result.changes === 0) return res.status(404).json({ message: 'Agendamento nao encontrado.' });

      logAudit(req.user.id, 'updated_status', 'appointments', req.params.id);
      return res.json({ appointment: appointmentById(req.params.id) });
    } catch (error) {
      return next(error);
    }
  });

  router.patch('/admin/tickets/:id', authRequired, adminRequired, (req, res, next) => {
    try {
      const input = statusSchema.parse(req.body);
      if (!['open', 'in_review', 'resolved'].includes(input.status)) {
        return res.status(400).json({ message: 'Status de chamado invalido.' });
      }

      const result = db.prepare('UPDATE support_tickets SET status = ? WHERE id = ?').run(input.status, req.params.id);
      if (result.changes === 0) return res.status(404).json({ message: 'Chamado nao encontrado.' });

      logAudit(req.user.id, 'updated_status', 'support_tickets', req.params.id);
      return res.json({ ok: true });
    } catch (error) {
      return next(error);
    }
  });

  router.patch('/admin/integrations/:id', authRequired, adminRequired, (req, res, next) => {
    try {
      const input = statusSchema.parse(req.body);
      if (!['online', 'degraded', 'offline'].includes(input.status)) {
        return res.status(400).json({ message: 'Status de integracao invalido.' });
      }

      const latency = Math.floor(90 + Math.random() * 720);
      const result = db
        .prepare('UPDATE integrations SET status = ?, last_sync = ?, latency_ms = ? WHERE id = ?')
        .run(input.status, now(), latency, req.params.id);
      if (result.changes === 0) return res.status(404).json({ message: 'Integracao nao encontrada.' });

      logAudit(req.user.id, 'updated_status', 'integrations', req.params.id);
      return res.json({ integration: db.prepare('SELECT * FROM integrations WHERE id = ?').get(req.params.id) });
    } catch (error) {
      return next(error);
    }
  });

  router.patch('/admin/triage/:id', authRequired, adminRequired, (req, res, next) => {
    try {
      const input = statusSchema.parse(req.body);
      if (!['waiting', 'in_service', 'resolved'].includes(input.status)) {
        return res.status(400).json({ message: 'Status de triagem invalido.' });
      }

      const result = db.prepare('UPDATE triage_cases SET status = ? WHERE id = ?').run(input.status, req.params.id);
      if (result.changes === 0) return res.status(404).json({ message: 'Triagem não encontrada.' });

      logAudit(req.user.id, 'updated_status', 'triage_cases', req.params.id);
      return res.json({ ok: true });
    } catch (error) {
      return next(error);
    }
  });

  router.patch('/admin/queue/:id', authRequired, adminRequired, (req, res, next) => {
    try {
      const input = statusSchema.parse(req.body);
      if (!['waiting', 'called', 'done', 'cancelled'].includes(input.status)) {
        return res.status(400).json({ message: 'Status de fila inválido.' });
      }

      const result = db.prepare('UPDATE queue_entries SET status = ? WHERE id = ?').run(input.status, req.params.id);
      if (result.changes === 0) return res.status(404).json({ message: 'Entrada de fila não encontrada.' });

      logAudit(req.user.id, 'updated_status', 'queue_entries', req.params.id);
      return res.json({ queue: queueById(req.params.id) });
    } catch (error) {
      return next(error);
    }
  });

  function appointmentsForUser(userId) {

```

```

    return db
      .prepare(`
        SELECT appointments.*, units.name AS unit_name, units.district AS unit_district, units.phone AS unit_phone
        FROM appointments
        JOIN units ON units.id = appointments.unit_id
        WHERE appointments.user_id = ?
        ORDER BY scheduled_at ASC
      `)
      .all(userId);
  }

function appointmentById(id) {
  return db
    .prepare(`
      SELECT appointments.*, users.name AS user_name, users.email AS user_email, units.name AS unit_name
      FROM appointments
      JOIN users ON users.id = appointments.user_id
      JOIN units ON units.id = appointments.unit_id
      WHERE appointments.id = ?
    `)
    .get(id);
}

function listUnits() {
  return db
    .prepare('SELECT * FROM units ORDER BY distance_km ASC, name ASC')
    .all()
    .map(normalizeUnit);
}

function normalizeUnit(unit) {
  return {
    ...unit,
    services: JSON.parse(unit.services),
    distance_km: Number(unit.distance_km),
    lat: Number(unit.lat),
    lng: Number(unit.lng),
  };
}

function isValidImageBuffer(buffer, mimeType) {
  if (mimeType === 'image/png') {
    return buffer.subarray(0, 8).equals(Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]));
  }

  if (mimeType === 'image/jpeg' || mimeType === 'image/jpg') {
    // Linhas 766-842
    return buffer[0] === 0xff && buffer[1] === 0xd8 && buffer[buffer.length - 2] === 0xff && buffer[buffer.length - 1] === 0;
  }

  if (mimeType === 'image/webp') {
    return buffer.subarray(0, 4).toString('ascii') === 'RIFF' && buffer.subarray(8, 12).toString('ascii') === 'WEBP';
  }

  return false;
}

function profileForUser(userId) {
  return (
    db.prepare('SELECT * FROM patient_profiles WHERE user_id = ?').get(userId) || {
      user_id: userId,
      cpf: '',
      birth_date: '',
      phone: '',
      sus_card: '',
      address: '',
      emergency_contact: '',
      updated_at: now(),
    }
  );
}

function triageForUser(userId) {
  return db.prepare('SELECT * FROM triage_cases WHERE user_id = ? ORDER BY created_at DESC').all(userId);
}

function queueForUser(userId) {
  return db
    .prepare(`
      SELECT queue_entries.*, units.name AS unit_name, units.address AS unit_address
      FROM queue_entries
      JOIN units ON units.id = queue_entries.unit_id
      WHERE queue_entries.user_id = ?
      ORDER BY queue_entries.status = 'waiting' DESC, queue_entries.created_at DESC
    `)
    .all(userId);
}

function queueById(id) {
  return db

```

```

        .prepare(`
        SELECT queue_entries.*, users.name AS user_name, units.name AS unit_name, units.address AS unit_address
        FROM queue_entries
        JOIN users ON users.id = queue_entries.user_id
        JOIN units ON units.id = queue_entries.unit_id
        WHERE queue_entries.id = ?
        `)
        .get(id);
    }

function recommendationForRisk(riskLevel) {
    return {
        low: 'Prioridade verde. Acompanhe sintomas e procure atendimento de rotina.',
        medium: 'Prioridade amarela. Procure uma unidade próxima ou aguarde chamada da equipe.',
        high: 'Prioridade laranja. Atendimento prioritário recomendado ainda hoje.',
        critical: 'Prioridade vermelha. Procure emergência imediatamente ou ligue 192.',
    }[riskLevel];
}

function count(table) {
    return db.prepare(`SELECT COUNT(*) AS total FROM ${table}`)().get().total;
}

function initials(name) {
    return name
        .split(/\s+/)
        .filter(Boolean)
        .slice(0, 2)
        .map((part) => part[0]?.toUpperCase())
        .join('');
}

export default router;

```

Arquivo: server/index.js

Servidor Express: middlewares (helmet, CORS, rate limit), montagem da API, arquivos estaticos dist/ e fallback SPA para producao.

Total: 115 linhas | Linguagem: .js

```
import 'dotenv/config';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import cookieParser from 'cookie-parser';
import cors from 'cors';
import express from 'express';
import rateLimit from 'express-rate-limit';
import helmet from 'helmet';
import morgan from 'morgan';
import passport from 'passport';
import router from './routes.js';
import { dataDir, initDb } from './db.js';

initDb();

const app = express();
const port = Number(process.env.PORT || 3001);
const __dirname = path.dirname(fileURLToPath(import.meta.url));
const distDir = path.resolve(__dirname, '..', 'dist');

const allowedOrigins = (
  process.env.CLIENT_ORIGIN ||
  `http://localhost:5173,http://localhost:${port},http://localhost,capacitor://localhost,ionic://localhost`
)
  .split(',')
  .map((origin) => origin.trim())
  .filter(Boolean);

const corsOptions = (req, callback) => {
  const origin = req.get('origin');

  if (!origin || allowedOrigins.includes(origin) || isSameHostOrigin(req, origin) || isDemoTunnelOrigin(origin)) {
    return callback(null, { origin: true, credentials: true });
  }

  return callback(new Error('Origem nao autorizada por CORS.'));
};

function isSameHostOrigin(req, origin) {
  try {
    const originUrl = new URL(origin);
    const hosts = [req.get('host'), req.get('x-forwarded-host')].filter(Boolean);
    return hosts.includes(originUrl.host);
  } catch {
    return false;
  }
}

function isDemoTunnelOrigin(origin) {
  try {
    const { hostname } = new URL(origin);
    return ['serveusercontent.com', '.localt', '.trycloudflare.com'].some((suffix) =>
      hostname.endsWith(suffix),
    );
  } catch {
    return false;
  }
}

app.set('trust proxy', 1);
app.disable('x-powered-by');
app.use(
  helmet({
    contentSecurityPolicy: false,
  }),
);
app.use(
  rateLimit({
    windowMs: 15 * 60 * 1000,
    limit: 300,
    standardHeaders: true,
    legacyHeaders: false,
  }),
);
app.use(express.json({ limit: '4mb' }));
app.use(cookieParser());
app.use(passport.initialize());
app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev'));

app.use('/api', cors(corsOptions), router);
app.use('/uploads', express.static(path.join(dataDir, 'uploads')));
app.use('/api/uploads', express.static(path.join(dataDir, 'uploads')));
app.use(express.static(distDir));
```

```
app.use((req, res, next) => {
  if (req.method === 'GET' && !req.path.startsWith('/api')) {
    return res.sendFile(path.join(distDir, 'index.html'), (error) => {
      if (error) next();
    });
  }
  return next();
});

app.use((error, _req, res, _next) => {
  if (error?.issues) {
    const firstMessage = error.issues.find((issue) => issue?.message)?.message;
    return res.status(400).json({ message: firstMessage || 'Dados invalidos.', details: error.issues });
  }

  if (error?.message?.includes('CORS')) {
    return res.status(403).json({ message: error.message });
  }

  console.error(error);
  return res.status(500).json({ message: 'Erro interno no servidor.' });
});

if (!process.env.VERCEL) {
  app.listen(port, () => {
    console.log('SaudeConnect API em http://localhost:${port}');
  });
}

export default app;
```

5. Deploy e infraestrutura

Adaptacao serverless, Docker e configuração Vercel.

Arquivo: api/[...path].js

Adaptador Vercel serverless que exporta o app Express para funcoes /api/.*

Total: 4 linhas | Linguagem: js

```
import app from '../server/index.js';  
  
export default app;
```

Arquivo: vercel.json

Deploy Vercel: build, rotas /api para serverless e fallback SPA index.html.

Total: 24 linhas | Linguagem: .json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "functions": {
    "api/[...path].js": {
      "maxDuration": 30
    }
  },
  "routes": [
    {
      "src": "/api/(.*)",
      "dest": "/api/[...path]"
    },
    {
      "handle": "filesystem"
    },
    {
      "src": "/(.*)",
      "dest": "/index.html"
    }
  ]
}
```


Arquivo: Dockerfile

Imagem Docker Node 24: npm ci, build e start na porta 7860 (Hugging Face).

Total: 18 linhas | Linguagem: texto

```
FROM node:24-bookworm-slim
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY . .
```

```
ENV NODE_ENV=production
```

```
ENV PORT=7860
```

```
RUN npm run build
```

```
EXPOSE 7860
```

```
CMD ["npm", "run", "start"]
```

Arquivo: .github/workflows/deploy-huggingface.yml

CI/CD para deploy no Hugging Face Spaces.

Total: 60 linhas | Linguagem: .yml

name: Deploy Hugging Face Space

```
on:
  workflow_dispatch:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      HF_SPACE_ID: ${vars.HF_SPACE_ID || 'Joaoemmanuel2666/SaudeConnect-Web' }}
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Install Hugging Face Hub
        run: python -m pip install --upgrade huggingface_hub

      - name: Create or update Space
        env:
          HF_TOKEN: ${secrets.HF_TOKEN}
        run: |
          python - <<'PY'
          import os
          from huggingface_hub import HfApi, create_repo, upload_folder

          token = os.environ["HF_TOKEN"]
          repo_id = os.environ["HF_SPACE_ID"]

          create_repo(
              repo_id=repo_id,
              repo_type="space",
              space_sdk="docker",
              private=False,
              exist_ok=True,
              token=token,
          )

          upload_folder(
              repo_id=repo_id,
              repo_type="space",
              folder_path=".",
              token=token,
              ignore_patterns=[
                  ".git/*",
                  "node_modules/*",
                  "dist/*",
                  "data/*",
                  "android/.gradle/*",
                  "android/build/*",
                  "android/app/build/*",
                  "artifacts/screenshots/*",
                  "*.log",
                  ".env",
              ],
          )
PY
```

6. Android (Capacitor)

Projeto nativo Android que empacota o site compilado.

Arquivo: android/app/src/main/java/com/saudeconnect/app/MainActivity.java

Activity Android Capacitor. Herda BridgeActivity para carregar o web app.

Total: 6 linhas | Linguagem: java

```
package com.saudeconnect.app;  
  
import com.getcapacitor.BridgeActivity;  
  
public class MainActivity extends BridgeActivity {}
```

Arquivo: android/app/src/main/AndroidManifest.xml

Manifest Android: permissões, activity principal e configuração do app.

Total: 42 linhas | Linguagem: .xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/AppTheme">

        <activity
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLay...
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:exported="true">

                <intent-filter>
                    <action android:name="android.intent.action.MAIN" />
                    <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>

            </activity>

            <provider
                android:name="androidx.core.content.FileProvider"
                android:authorities="${applicationId}.fileprovider"
                android:exported="false"
                android:grantUriPermissions="true">
                <meta-data
                    android:name="android.support.FILE_PROVIDER_PATHS"
                    android:resource="@xml/file_paths"></meta-data>
            </provider>
        </application>

        <!-- Permissions -->

        <uses-permission android:name="android.permission.INTERNET" />
    </manifest>
```

Arquivo: android/app/src/main/res/layout/activity_main.xml

Recurso ou configuração XML Android.

Total: 13 linhas | Linguagem: .xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <WebView
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Arquivo: android/app/build.gradle

Arquivo Gradle de build Android.

Total: 55 linhas | Linguagem: .gradle

```
apply plugin: 'com.android.application'

android {
    namespace = "com.saudeconnect.app"
    compileSdk = rootProject.ext.compileSdkVersion
    defaultConfig {
        applicationId "com.saudeconnect.app"
        minSdkVersion rootProject.ext.minSdkVersion
        targetSdkVersion rootProject.ext.targetSdkVersion
        versionCode 1
        versionName "1.0"
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        aaptOptions {
            // Files and dirs to omit from the packaged assets dir, modified to accommodate modern web apps.
            // Default: https://android.googlesource.com/platform/frameworks/base/+282e181b58cf72b6ca770dc7ca5f91f1...
            ignoreAssetsPattern = '!svn:!git:!ds_store:!*.scc:.*:!CVS:!thumbs.db:!picasa.ini:!*~'
        }
    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
}

repositories {
    flatDir{
        dirs '../capacitor-cordova-android-plugins/src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "androidx.coordinatorlayout:coordinatorlayout:$androidxCoordinatorLayoutVersion"
    implementation "androidx.core:core-splashscreen:$coreSplashScreenVersion"
    implementation project(':capacitor-android')
    testImplementation "junit:junit:$junitVersion"
    androidTestImplementation "androidx.test.ext:junit:$androidxJUnitVersion"
    androidTestImplementation "androidx.test.espresso:espresso-core:$androidxEspressoCoreVersion"
    implementation project(':capacitor-cordova-android-plugins')
}

apply from: 'capacitor.build.gradle'

try {
    def servicesJSON = file('google-services.json')
    if (servicesJSON.text) {
        apply plugin: 'com.google.gms.google-services'
    }
} catch (Exception e) {
    logger.info("google-services.json not found, google-services plugin not applied. Push Notifications won't work")
}
```

Arquivo: android/app/capacitor.build.gradle

Arquivo Gradle de build Android.

Total: 20 linhas | Linguagem: .gradle

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
```

```
android {  
    compileOptions {  
        sourceCompatibility JavaVersion.VERSION_21  
        targetCompatibility JavaVersion.VERSION_21  
    }  
}  
  
apply from: "../capacitor-cordova-android-plugins/cordova.variables.gradle"  
dependencies {  
  
}  
  
if (hasProperty('postBuildExtras')) {  
    postBuildExtras()  
}
```

Arquivo: android/build.gradle

Arquivo Gradle de build Android.

Total: 30 linhas | Linguagem: .gradle

// Top-level build file where you can add configuration options common to all sub-projects/modules.

```
buildscript {  
    repositories {  
        google()  
        mavenCentral()  
    }  
    dependencies {  
        classpath 'com.android.tools.build:gradle:8.13.0'  
        classpath 'com.google.gms:google-services:4.4.4'  
  
        // NOTE: Do not place your application dependencies here; they belong  
        // in the individual module build.gradle files  
    }  
}  
  
apply from: "variables.gradle"  
  
allprojects {  
    repositories {  
        google()  
        mavenCentral()  
    }  
}  
  
task clean(type: Delete) {  
    delete rootProject.buildDir  
}
```


Arquivo: android/settings.gradle

Arquivo Gradle de build Android.

Total: 5 linhas | Linguagem: .gradle

```
include ':app'  
include ':capacitor-cordova-android-plugins'  
project(':capacitor-cordova-android-plugins').projectDir = new File('./capacitor-cordova-android-plugins/')  
  
apply from: 'capacitor.settings.gradle'
```

Arquivo: android/capacitor.settings.gradle

Arquivo Gradle de build Android.

Total: 4 linhas | Linguagem: .gradle

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
include ':capacitor-android'
project(':capacitor-android').projectDir = new File('../node_modules/@capacitor/android/capacitor')
```

Arquivo: android/variables.gradle

Arquivo Gradle de build Android.

Total: 16 linhas | Linguagem: .gradle

```
ext {  
    minSdkVersion = 24  
    compileSdkVersion = 36  
    targetSdkVersion = 36  
    androidxActivityVersion = '1.11.0'  
    androidxAppCompatVersion = '1.7.1'  
    androidxCoordinatorLayoutVersion = '1.3.0'  
    androidxCoreVersion = '1.17.0'  
    androidxFragmentVersion = '1.8.9'  
    coreSplashScreenVersion = '1.2.0'  
    androidxWebkitVersion = '1.14.0'  
    junitVersion = '4.13.2'  
    androidxJUnitVersion = '1.3.0'  
    androidxEspressoCoreVersion = '3.7.0'  
    cordovaAndroidVersion = '14.0.1'  
}
```

Arquivo: android/gradle.properties

Propriedades de build Gradle.

Total: 23 linhas | Linguagem: .properties

```
# Project-wide Gradle settings.

# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.

# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html

# Specifies the JVM arguments used for the daemon process.
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx1536m

# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. More details, visit
# http://www.gradle.org/docs/current/userguide/multi_project_builds.html#sec:decoupled_projects
# org.gradle.parallel=true

# AndroidX package structure to make it clearer which packages are bundled with the
# Android operating system, and which are packaged with your app's APK
# https://developer.android.com/topic/libraries/support-library/androidx-rn
android.useAndroidX=true
```

7. Scripts auxiliares

Automatizacao de testes visuais e capturas de tela.

Arquivo: scripts/smoke-visual.mjs

Teste visual automatizado com Puppeteer.

Total: 90 linhas | Linguagem: .mjs

```
import fs from 'node:fs';
import path from 'node:path';
import puppeteer from 'puppeteer-core';

const baseUrl = process.env.APP_URL || 'http://localhost:3001';
const chromePath =
  process.env.CHROME_PATH || 'C:/Program Files/Google/Chrome/Application/chrome.exe';
const outputDir = path.resolve(process.cwd(), 'artifacts', 'screenshots');
fs.mkdirSync(outputDir, { recursive: true });

const browser = await puppeteer.launch({
  executablePath: chromePath,
  headless: true,
  args: ['--no-sandbox', '--disable-setuid-sandbox'],
});

const page = await browser.newPage();
const issues = [];
page.on('pageerror', (error) => issues.push(error.message));
page.on('console', (message) => {
  if (['error', 'warning'].includes(message.type())) issues.push(message.text());
});
page.on('requestfailed', (request) => {
  issues.push(`Falha ao carregar ${request.url()}: ${request.failure()?.errorText}`);
});

async function save(name) {
  const filePath = path.join(outputDir, name);
  await page.screenshot({ path: filePath, fullPage: true });
  return filePath;
}

async function hasOverflow() {
  return page.evaluate(
    () => document.documentElement.scrollWidth > document.documentElement.clientWidth + 2,
  );
}

await page.setViewport({ width: 1440, height: 1000, deviceScaleFactor: 1 });
await page.goto(`${baseUrl}/login`, { waitUntil: 'networkidle0' });
await page.evaluate(() => localStorage.clear());
await page.reload({ waitUntil: 'networkidle0' });
try {
  await page.waitForSelector('.auth-shell', { timeout: 15000 });
} catch (error) {
  fs.writeFileSync(path.join(outputDir, 'debug.html'), await page.content(), 'utf8');
  throw new Error(
    `Interface nao montou. URL=${page.url()} Issues=${issues.join(' | ')} || 'sem erros capturados' Cause=${error.mes...
  );
}
await save('login-desktop.png');
if (await hasOverflow()) issues.push('Overflow horizontal na tela de login desktop.');
```

```
await page.type('input[type="email"]', 'paciente@saudeconnect.com');
await page.type('input[type="password"]', 'Paciente@12345');
await page.click('.primary-action');
await page.waitForSelector('.app-shell', { timeout: 15000 });
await save('paciente-desktop.png');
if (await hasOverflow()) issues.push('Overflow horizontal no painel do paciente desktop.');
```

```
await page.goto(`${baseUrl}/app/mapa`, { waitUntil: 'networkidle0' });
await page.waitForSelector('.leaflet-map .leaflet-marker-icon, .health-marker', { timeout: 15000 });
await save('mapa-desktop.png');
if (await hasOverflow()) issues.push('Overflow horizontal na pagina de mapa desktop.');
```

```
await page.goto(`${baseUrl}/app`, { waitUntil: 'networkidle0' });
await page.setViewport({ width: 390, height: 900, isMobile: true, deviceScaleFactor: 2 });
await page.waitForSelector('.metric-card', { timeout: 15000 });
await save('paciente-mobile.png');
if (await hasOverflow()) issues.push('Overflow horizontal no painel do paciente mobile.');
```

```
await page.evaluate(() => localStorage.clear());
await page.setViewport({ width: 1440, height: 1000, deviceScaleFactor: 1 });
await page.goto(`${baseUrl}/login`, { waitUntil: 'networkidle0' });
await page.type('input[type="email"]', 'admin@saudeconnect.com');
await page.type('input[type="password"]', 'Admin@12345');
await page.click('.primary-action');
await page.waitForSelector('.admin-panel', { timeout: 15000 });
await save('admin-desktop.png');
if (await hasOverflow()) issues.push('Overflow horizontal no painel admin desktop.');
```

```
await browser.close();

if (issues.length) {
```

```
    console.error(issues.join('\n'));
    process.exit(1);
}

console.log(`Capturas salvas em ${outputDir}`);
```

Arquivo: scripts/generate-marketing-screenshots.mjs

Gera capturas de tela para marketing.

Total: 199 linhas | Linguagem: .mjs

```
import fs from 'node:fs';
import path from 'node:path';
import puppeteer from 'puppeteer-core';

const baseUrl = process.env.APP_URL || 'http://localhost:3001';
const chromePath =
  process.env.CHROME_PATH || 'C:/Program Files/Google/Chrome/Application/chrome.exe';
const docsDir = path.resolve(process.cwd(), 'docs', 'screenshots');
const playDir = path.resolve(process.cwd(), 'store-assets', 'google-play');
const logPath = path.resolve(process.cwd(), 'artifacts', 'screenshots-generation.log');

fs.mkdirSync(docsDir, { recursive: true });
fs.mkdirSync(playDir, { recursive: true });
fs.mkdirSync(path.dirname(logPath), { recursive: true });
fs.writeFileSync(logPath, '');

const screens = [
  {
    slug: 'login',
    title: 'Login',
    path: '/login',
    auth: null,
    waitFor: '.auth-shell',
  },
  {
    slug: 'inicio',
    title: 'Inicio do paciente',
    path: '/app',
    auth: 'patient',
    waitFor: '.metric-card',
  },
  {
    slug: 'mapa',
    title: 'Mapa de unidades',
    path: '/app/mapa',
    auth: 'patient',
    waitFor: '.leaflet-map',
    afterLoad: async (page) => {
      await page
        .waitForSelector('.leaflet-marker-icon, .health-marker', { timeout: 10000 })
        .catch(() => undefined);
      await page.click('.leaflet-marker-icon, .health-marker').catch(() => undefined);
      await sleep(700);
    },
  },
  {
    slug: 'cadastro',
    title: 'Cadastro do cidadao',
    path: '/app/cadastro',
    auth: 'patient',
    waitFor: '.photo-uploader',
  },
  {
    slug: 'prontuarios',
    title: 'Prontuarios',
    path: '/app/prontuarios',
    auth: 'patient',
    waitFor: '.timeline',
  },
  {
    slug: 'triagem',
    title: 'Triagem',
    path: '/app/triagem',
    auth: 'patient',
    waitFor: '.action-panel',
  },
  {
    slug: 'fila',
    title: 'Fila',
    path: '/app/fila',
    auth: 'patient',
    waitFor: '.queue-board',
  },
  {
    slug: 'admin',
    title: 'Painel administrativo',
    path: '/admin',
    auth: 'admin',
    waitFor: '.admin-panel',
  },
];

const browser = await puppeteer.launch({
  executablePath: chromePath,
```

```

    headless: true,
    args: ['--no-sandbox', '--disable-setuid-sandbox'],
  });
  progress('Chrome iniciado.');
```

```

const issues = [];
const page = await browser.newPage();
progress('Pagina de captura preparada.');
```

```

page.on('pageerror', (error) => issues.push(error.message));
page.on('console', (message) => {
  if (['error', 'warning'].includes(message.type())) issues.push(message.text());
});
page.on('requestfailed', (request) => {
  const url = request.url();
  if (!url.includes('tile.openstreetmap.org')) {
    issues.push(`Falha ao carregar ${url}: ${request.failure()?.errorText}`);
  }
});
```

```

const tokens = {
  patient: await login('paciente@saudeconnect.com', 'Paciente@12345'),
  admin: await login('admin@saudeconnect.com', 'Admin@12345'),
};
progress('Sesoes de teste autenticadas.');
```

```

for (const [index, screen] of screens.entries()) {
  const prefix = `${String(index + 1).padStart(2, '0')}-${screen.slug}`;

  await page.setViewport({ width: 1440, height: 1000, deviceScaleFactor: 1, isMobile: false });
  await captureScreen(page, screen, {
    path: path.join(docsDir, `${prefix}-desktop.png`),
    fullPage: false,
  });

  await page.setViewport({
    width: 360,
    height: 640,
    deviceScaleFactor: 3,
    isMobile: true,
  });
  await captureScreen(page, screen, {
    path: path.join(playDir, `${prefix}-${screen.slug}.jpg`),
    fullPage: false,
    type: 'jpeg',
    quality: 94,
  });
}

await browser.close();

if (issues.length) {
  console.error(issues.join('\n'));
  process.exit(1);
}

console.log(`Capturas para README salvas em ${docsDir}`);
console.log(`Capturas para Play Store salvas em ${playDir}`);

async function login(email, password) {
  const response = await fetch(`${baseUrl}/api/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });

  if (!response.ok) {
    throw new Error(`Falha ao autenticar ${email}: ${await response.text()}`);
  }

  const payload = await response.json();
  return payload.token;
}

async function captureScreen(page, screen, options) {
  progress(`Abrindo ${screen.title} para ${path.basename(options.path)}...`);
  if (!screen.auth) {
    await page.goto(`${baseUrl}/login`, { waitUntil: 'domcontentloaded' });
    await page.evaluate(() => localStorage.clear());
  } else {
    if (!page.url().startsWith(baseUrl)) {
      await page.goto(`${baseUrl}/login`, { waitUntil: 'domcontentloaded' });
    }
    await page.evaluate((token) => localStorage.setItem('saudeconnect.token', token), tokens[screen.auth]);
  }

  await page.goto(`${baseUrl}${screen.path}`, { waitUntil: 'domcontentloaded' });
  await page.waitForSelector(screen.waitFor, { timeout: 15000 });
  await sleep(700);

  if (screen.afterLoad) {

```



```
    await screen.afterLoad(page);
  }

  await page.screenshot(options);
  progress(`Arquivo salvo: ${options.path}`);

  const hasOverflow = await page.evaluate(
    () => document.documentElement.scrollWidth > document.documentElement.clientWidth + 2,
  );
  if (hasOverflow) {
    issues.push(`Overflow horizontal em ${screen.title}.`);
  }
}

function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function progress(message) {
  const line = `[${new Date().toISOString()}] ${message}`;
  console.log(line);
  fs.appendFileSync(logPath, `${line}\n`);
}
```

8. Demais arquivos fonte

Arquivos adicionais do repositório incluídos integralmente.

Arquivo: .dockerignore

Arquivo fonte .dockerignore.

Total: 11 linhas

```
node_modules
dist
data
artifacts
android/.gradle
android/build
android/app/build
*.log
.env
.git
```

Arquivo: .gitattributes

Arquivo fonte .gitattributes.

Total: 7 linhas

```
*.pdf binary  
*.apk binary  
*.APK binary  
*.png binary  
*.jpg binary  
*.jpeg binary
```

Arquivo: .gitignore

Arquivo fonte .gitignore.

Total: 34 linhas

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local
.env
data
artifacts/screenshots
android/.gradle
android/build
android/app/build
android/local.properties

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?

.vercel
```

Arquivo: .vercel/project.json

Arquivo de configuração JSON.

Total: 1 linhas

```
{"projectId": "prj_BEAApdoM2xkrbKfmYFnFZeHnm0K", "orgId": "team_21cuvuABDfyFMQMpaggD32kr", "projectName": "saudeconnectl"}
```

Arquivo: android/.gitignore

Arquivo fonte android/.gitignore.

Total: 102 linhas

```
# Using Android gitignore template: https://github.com/github/gitignore/blob/HEAD/Android.gitignore

# Built application files
*.apk
*.aar
*.ap_
*.aab

# Files for the ART/Dalvik VM
*.dex

# Java class files
*.class

# Generated files
bin/
gen/
out/
# Uncomment the following line in case you need and you don't have the release build type files in your app
# release/

# Gradle files
.gradle/
build/

# Local configuration file (sdk path, etc)
local.properties

# Proguard folder generated by Eclipse
proguard/

# Log Files
*.log

# Android Studio Navigation editor temp files
.navigation/

# Android Studio captures folder
captures/

# IntelliJ
*.iml
.idea/workspace.xml
.idea/tasks.xml
.idea/gradle.xml
.idea/assetWizardSettings.xml
.idea/dictionaries
.idea/libraries
# Android Studio 3 in .gitignore file.
.idea/caches
.idea/modules.xml
# Comment next line if keeping position of elements in Navigation Editor is relevant for you
.idea/navEditor.xml

# Keystore files
# Uncomment the following lines if you do not want to check your keystore files in.
#*.jks
#*.keystore

# External native build folder generated in Android Studio 2.2 and later
.externalNativeBuild
.cxx/

# Google Services (e.g. APIs or Firebase)
# google-services.json

# Freeline
freeline.py
freeline/
freeline_project_description.json

# fastlane
fastlane/report.xml
fastlane/Preview.html
fastlane/screenshots
fastlane/test_output
fastlane/readme.md

# Version control
vcs.xml

# lint
lint/intermediates/
lint/generated/
```

```
lint/outputs/  
lint/tmp/  
# lint/reports/  
  
# Android Profiling  
*.hprof  
  
# Cordova plugins for Capacitor  
capacitor-cordova-android-plugins  
  
# Copied web assets  
app/src/main/assets/public  
  
# Generated Config files  
app/src/main/assets/capacitor.config.json  
app/src/main/assets/capacitor.plugins.json  
app/src/main/res/xml/config.xml
```


Arquivo: android/app/.gitignore

Arquivo fonte android/app/.gitignore.

Total: 3 linhas

```
/build/*  
!/build/.npmkeep
```

Arquivo: android/app/proguard-rules.pro

Arquivo fonte android/app/proguard-rules.pro.

Total: 22 linhas

```
# Add project specific ProGuard rules here.
# You can control the set of applied configuration files using the
# proguardFiles setting in build.gradle.
#
# For more details, see
#   http://developer.android.com/guide/developing/tools/proguard.html

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#   public *;
#}

# Uncomment this to preserve the line number information for
# debugging stack traces.
#-keepattributes SourceFile,LineNumberTable

# If you keep the line number information, uncomment this to
# hide the original source file name.
#-renamesourcefileattribute SourceFile
```

Arquivo:

android/app/src/androidTest/java/com/getcapacitor/myapp/ExampleInstrumentedTest.java

Código Java Android.

Total: 27 linhas

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;

import android.content.Context;
import androidx.test.ext.junit.runners.AndroidJUnit4;
import androidx.test.platform.app.InstrumentationRegistry;
import org.junit.Test;
import org.junit.runner.RunWith;

/**
 * Instrumented test, which will execute on an Android device.
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
@RunWith(AndroidJUnit4.class)
public class ExampleInstrumentedTest {

    @Test
    public void useAppContext() throws Exception {
        // Context of the app under test.
        Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();

        assertEquals("com.getcapacitor.app", appContext.getPackageName());
    }
}
```

Arquivo: android/app/src/main/assets/capacitor.config.json

Arquivo de configuração JSON.

Total: 9 linhas

```
{
  ■ "appId": "com.saudeconnect.app",
  ■ "appName": "SaudeConnect",
  ■ "webDir": "dist",
  ■ "server": {
  ■ ■ "androidScheme": "https"
  ■ }
}
```

Arquivo: android/app/src/main/assets/capacitor.plugins.json

Arquivo de configuração JSON.

Total: 2 linhas

```
[ ]
```

Arquivo: android/app/src/main/assets/public/assets/index-BjBfqSh.js

Arquivo fonte android/app/src/main/assets/public/assets/index-BjBfqSh.js.

Total: 11 linhas

```
// Linhas 1-2
var e=Object.create,t=Object.defineProperty,n=Object.getOwnPropertyDescriptor,r=Object.getOwnPropertyNames,i=Object.g...

// Linhas 3-9
    at`)?`(<anonymous>`):-1<e.stack.indexOf(`@`)?`@unknown:0:0:`}`return`
`+we+e+Te}var De=!1;function Oe(e,t){if(!e||De)return``;De=!0;var n=Error.prepareStackTrace;Error.prepareStackTrace=v...
`),l=s.split(`
`);for(i=r=0;r<c.length&&!c[r].includes(`DetermineComponentFrameRoot`);)r++;for(;i<l.length&&!l[i].includes(`Determin...
`+c[r].replace(` at new `,` at `);return e.displayName&&u.includes(`<anonymous>`)&&(u=u.replace(`<anonymous>`,e.displ...
Error generating stack: `+e.message+`

// Linhas 10-11
`+e.stack}}var je=Object.prototype.hasOwnProperty,Me=t.unstable_scheduleCallback,Ne=t.unstable_cancelCallback,Pe=t.un...

// Linhas 12-13
`).replace(Ad,``)}function Md(e,t){return t=jd(t),jd(e)==t}function $(e,t,n,r,a,o){switch(n){case`children`:typeof r...

// Linhas 14-15

// Linhas 16-16
Please change the parent <Route path="$${e}"> to <Route path="$${e}===`/?*`:`${e}/`">.`)}let u=qt(),d;if(t){let e=ty...
```

Arquivo: android/app/src/main/assets/public/assets/index-PwuHWK31.css

Arquivo fonte android/app/src/main/assets/public/assets/index-PwuHWK31.css.

Total: 2 linhas

```
:root{--bg:#f5f7fb;--surface:#fff;--surface-strong:#eef6f7;--text:#1f2937;--muted:#697586;--border:#dde5ee;--line:#ed...
```

Arquivo: android/app/src/main/assets/public/cordova.js

Arquivo fonte android/app/src/main/assets/public/cordova.js.

Total: 1 linhas

Arquivo: android/app/src/main/assets/public/cordova_plugins.js

Arquivo fonte android/app/src/main/assets/public/cordova_plugins.js.

Total: 1 linhas

Arquivo: android/app/src/main/assets/public/favicon.svg

Arquivo fonte android/app/src/main/assets/public/favicon.svg.

Total: 5 linhas

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256">
  <rect width="256" height="256" rx="58" fill="#0f8b8d"/>
  <path fill="ffff" d="M128 56c-29.8 0-54 24.2-54 0 47.8 54 90s54-42.2 54-90c0-29.8-24.2-54-54-54Zm0 26c15.5...
```

Arquivo: android/app/src/main/assets/public/icons.svg

Arquivo fonte android/app/src/main/assets/public/icons.svg.

Total: 25 linhas

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="bluesky-icon" viewBox="0 0 16 17">
    <g clip-path="url(#bluesky-clip)"><path fill="#08060d" d="M7.75 7.735c-.693-1.348-2.58-3.86-4.334-5.097-1.68-1.18...
    <defs><clipPath id="bluesky-clip"><path fill="#fff" d="M.1.85h15.3v15.3H.1z"/></clipPath></defs>
  </symbol>
  <symbol id="discord-icon" viewBox="0 0 20 19">
    <path fill="#08060d" d="M16.224 3.768a14.5 14.5 0 0 0-3.67-1.153c-.158.286-.343.67-.47.976a13.5 13.5 0 0 0-4.067 ...
  </symbol>
  <symbol id="documentation-icon" viewBox="0 0 21 20">
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="m15.5 13...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M17.167 ...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M3 10a2...
  </symbol>
  <symbol id="github-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M9.356 1.85C5.05 1.85 1.57 5.356 1.57 9.694a7.84 7.84 0 0 0 5.324 7.4...
  </symbol>
  <symbol id="social-icon" viewBox="0 0 20 20">
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M12.5 6...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M2.5 16...
  </symbol>
  <symbol id="x-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M1.893 1.98c.052.072 1.245 1.769 2.653 3.771l2.892 4.114c.183.261.333...
  </symbol>
</svg>
```

Arquivo: android/app/src/main/assets/public/index.html

Arquivo fonte android/app/src/main/assets/public/index.html.

Total: 21 linhas

```
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <link rel="manifest" href="/manifest.webmanifest" />
    <meta name="theme-color" content="#0f8b8d" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta
      name="description"
      content="SaudeConnect: portal integrado para pacientes, unidades de saude e administradores."
    />
    <title>SaudeConnect</title>
    <script type="module" crossorigin src="/assets/index-BjBfqpSh.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-PwuHWK31.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

Arquivo: android/app/src/main/assets/public/manifest.webmanifest

Arquivo fonte android/app/src/main/assets/public/manifest.webmanifest.

Total: 18 linhas

```
{
  "name": "SaudeConnect",
  "short_name": "SaudeConnect",
  "description": "Portal integrado para pacientes, unidades de saude e administradores.",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#f5f7fb",
  "theme_color": "#0f8b8d",
  "icons": [
    {
      "src": "/favicon.svg",
      "sizes": "any",
      "type": "image/svg+xml",
      "purpose": "any maskable"
    }
  ]
}
```

Arquivo: android/app/src/main/res/drawable-v24/ic_launcher_foreground.xml

Recurso ou configuração XML Android.

Total: 35 linhas

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108">
    <path
        android:fillType="evenOdd"
        android:pathData="M32,64C32,64 38.39,52.99 44.13,50.95C51.37,48.37 70.14,49.57 70.14,49.57L108.26,87.69L108,1.11
        android:strokeColor="#00000000"
        android:strokeWidth="1">
        <aapt:attr name="android:fillColor">
            <gradient
                android:endX="78.5885"
                android:endY="90.9159"
                android:startX="48.7653"
                android:startY="61.0927"
                android:type="linear">
                <item
                    android:color="#44000000"
                    android:offset="0.0" />
                <item
                    android:color="#00000000"
                    android:offset="1.0" />
            </gradient>
        </aapt:attr>
    </path>
    <path
        android:fillColor="#FFFFFF"
        android:fillType="nonZero"
        android:pathData="M66.94,46.02L66.94,46.02C72.44,50.07 76,56.61 76,64L32,64C32,64 38.39,52.99 44.13,50.95
        android:strokeColor="#00000000"
        android:strokeWidth="1" />
</vector>
```

Arquivo: android/app/src/main/res/drawable/ic_launcher_background.xml

Recurso ou configuração XML Android.

Total: 171 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108">
    <path
        android:fillColor="#26A69A"
        android:pathData="M0,0h108v108h-108z" />
    <path
        android:fillColor="#00000000"
        android:pathData="M9,0L9,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,0L19,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M29,0L29,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M39,0L39,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M49,0L49,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,0L59,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,0L69,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,0L79,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M89,0L89,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M99,0L99,108"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,9L108,9"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,19L108,19"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,29L108,29"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,39L108,39"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,49L108,49"
        android:strokeColor="#33FFFFFF"
        android:strokeWidth="0.8" />
```

```
<path
  android:fillColor="#00000000"
  android:pathData="M0,59L108,59"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,69L108,69"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,79L108,79"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,89L108,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M0,99L108,99"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,29L89,29"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,39L89,39"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,49L89,49"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,59L89,59"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,69L89,69"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M19,79L89,79"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M29,19L29,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M39,19L39,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M49,19L49,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M59,19L59,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M69,19L69,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
<path
  android:fillColor="#00000000"
  android:pathData="M79,19L79,89"
  android:strokeColor="#33FFFFFF"
  android:strokeWidth="0.8" />
</vector>
```


Arquivo: android/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

Recurso ou configuração XML Android.

Total: 5 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@color/ic_launcher_background"/>
  <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

Arquivo: android/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

Recurso ou configuração XML Android.

Total: 5 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@color/ic_launcher_background"/>
  <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

Arquivo: android/app/src/main/res/values/ic_launcher_background.xml

Recurso ou configuração XML Android.

Total: 4 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="ic_launcher_background">#FFFFFF</color>
</resources>
```

Arquivo: android/app/src/main/res/values/strings.xml

Recurso ou configuração XML Android.

Total: 8 linhas

```
<?xml version='1.0' encoding='utf-8'?>
<resources>
    <string name="app_name">SaudeConnect</string>
    <string name="title_activity_main">SaudeConnect</string>
    <string name="package_name">com.saudeconnect.app</string>
    <string name="custom_url_scheme">com.saudeconnect.app</string>
</resources>
```

Arquivo: android/app/src/main/res/values/styles.xml

Recurso ou configuração XML Android.

Total: 22 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <!-- Base application theme. -->
    <style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
        <!-- Customize your theme here. -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>
    </style>

    <style name="AppTheme.NoActionBar" parent="Theme.AppCompat.DayNight.NoActionBar">
        <item name="windowActionBar">false</item>
        <item name="windowNoTitle">true</item>
        <item name="android:background">@null</item>
    </style>

    <style name="AppTheme.NoActionBarLaunch" parent="Theme.SplashScreen">
        <item name="android:background">@drawable/splash</item>
    </style>
</resources>
```

Arquivo: android/app/src/main/res/xml/config.xml

Recurso ou configuração XML Android.

Total: 6 linhas

```
<?xml version='1.0' encoding='utf-8'?>
<widget version="1.0.0" xmlns="http://www.w3.org/ns/widgets" xmlns:cdv="http://cordova.apache.org/ns/1.0">
  <access origin="*" />

</widget>
```

Arquivo: android/app/src/main/res/xml/file_paths.xml

Recurso ou configuração XML Android.

Total: 5 linhas

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
  <external-path name="my_images" path="." />
  <cache-path name="my_cache_images" path="." />
</paths>
```

Arquivo: android/app/src/test/java/com/getcapacitor/myapp/ExampleUnitTest.java

Código Java Android.

Total: 19 linhas

```
package com.getcapacitor.myapp;

import static org.junit.Assert.*;

import org.junit.Test;

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
public class ExampleUnitTest {

    @Test
    public void addition_isCorrect() throws Exception {
        assertEquals(4, 2 + 2);
    }
}
```


Arquivo: android/capacitor-cordova-android-plugins/build.gradle

Arquivo Gradle de build Android.

Total: 59 linhas

```

ext {
    androidxAppCompatVersion = project.hasProperty('androidxAppCompatVersion') ? rootProject.ext.androidxAppCompatVer...
    cordovaAndroidVersion = project.hasProperty('cordovaAndroidVersion') ? rootProject.ext.cordovaAndroidVersion : '1...'
}

buildscript {
    repositories {
        google()
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:8.13.0'
    }
}

apply plugin: 'com.android.library'

android {
    namespace = "capacitor.cordova.android.plugins"
    compileSdk = project.hasProperty('compileSdkVersion') ? rootProject.ext.compileSdkVersion : 36
    defaultConfig {
        minSdkVersion project.hasProperty('minSdkVersion') ? rootProject.ext.minSdkVersion : 24
        targetSdkVersion project.hasProperty('targetSdkVersion') ? rootProject.ext.targetSdkVersion : 36
        versionCode 1
        versionName "1.0"
    }
    lintOptions {
        abortOnError = false
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_21
        targetCompatibility JavaVersion.VERSION_21
    }
}

repositories {
    google()
    mavenCentral()
    flatDir{
        dirs 'src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(dir: 'src/main/libs', include: ['*.jar'])
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "org.apache.cordova:framework:$cordovaAndroidVersion"
    // SUB-PROJECT DEPENDENCIES START

    // SUB-PROJECT DEPENDENCIES END
}

// PLUGIN GRADLE EXTENSIONS START
apply from: "cordova.variables.gradle"
// PLUGIN GRADLE EXTENSIONS END

for (def func : cdvPluginPostBuildExtras) {
    func()
}

```

Arquivo: android/capacitor-cordova-android-plugins/cordova.variables.gradle

Arquivo Gradle de build Android.

Total: 7 linhas

```
// DO NOT EDIT THIS FILE! IT IS GENERATED EACH TIME "capacitor update" IS RUN
ext {
    cdvMinSdkVersion = project.hasProperty('minSdkVersion') ? rootProject.ext.minSdkVersion : 24
    // Plugin gradle extensions can append to this to have code run at the end.
    cdvPluginPostBuildExtras = []
    cordovaConfig = [:]
}
```

Arquivo: android/capacitor-cordova-android-plugins/src/main/AndroidManifest.xml

Recurso ou configuração XML Android.

Total: 8 linhas

```
<?xml version='1.0' encoding='utf-8'?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:amazon="http://schemas.amazon.com/apk/res/android">
<application >

</application>

</manifest>
```

Arquivo: android/gradlew.bat

Arquivo fonte android/gradlew.bat.

Total: 95 linhas

```
@rem
@rem Copyright 2015 the original author or authors.
@rem
@rem Licensed under the Apache License, Version 2.0 (the "License");
@rem you may not use this file except in compliance with the License.
@rem You may obtain a copy of the License at
@rem
@rem      https://www.apache.org/licenses/LICENSE-2.0
@rem
@rem Unless required by applicable law or agreed to in writing, software
@rem distributed under the License is distributed on an "AS IS" BASIS,
@rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
@rem See the License for the specific language governing permissions and
@rem limitations under the License.
@rem
@rem SPDX-License-Identifier: Apache-2.0
@rem

@if "%DEBUG%"=="" @echo off
@rem #####
@rem
@rem  Gradle startup script for Windows
@rem
@rem #####

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

set DIRNAME=%~dp0
if "%DIRNAME%"=="" set DIRNAME=.
@rem This is normally unused
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Resolve any "." and ".." in APP_HOME to make it shorter.
for %%i in ("%APP_HOME%") do set APP_HOME=%%~fi

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this script.
set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if %ERRORLEVEL% equ 0 goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH. 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:execute
@rem Setup the command line

set CLASSPATH=

@rem Execute Gradle
"%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname=%APP_BASE_NAME%" -classpath "%CLASSPA...

:end
@rem End local scope for the variables with windows NT shell
if %ERRORLEVEL% equ 0 goto mainEnd

:fail
rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code instead of
```

```
rem the _cmd.exe /c_ return code!
set EXIT_CODE=%ERRORLEVEL%
if %EXIT_CODE% equ 0 set EXIT_CODE=1
if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
exit /b %EXIT_CODE%

:mainEnd
if "%OS%"=="Windows_NT" endlocal

:omega
```

Arquivo: android/local.properties

Propriedades de build Gradle.

Total: 2 linhas

```
sdk.dir=C:/Users/joaoe/AppData/Local/Android/Sdk
```

Arquivo: artifacts/screenshots/debug.html

Arquivo fonte artifacts/screenshots/debug.html.

Total: 16 linhas

```
<!DOCTYPE html><html lang="pt-BR"><head>
  <meta charset="UTF-8">
  <link rel="icon" type="image/svg+xml" href="/favicon.svg">
  <link rel="manifest" href="/manifest.webmanifest">
  <meta name="theme-color" content="#0f8b8d">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="SaudeConnect: portal integrado para pacientes, unidades de saude e administrado...
  <title>SaudeConnect</title>
  <script type="module" crossorigin="" src="/assets/index-B1xvxGnB.js"></script>
  <link rel="stylesheet" crossorigin="" href="/assets/index-DUbSsbQu.css">
</head>
<body>
  <div id="root"></div>

</body></html>
```

Arquivo: public/favicon.svg

Arquivo fonte public/favicon.svg.

Total: 5 linhas

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 256 256">
  <rect width="256" height="256" rx="58" fill="#0f8b8d"/>
  <path fill="#fff" d="M128 56c-29.8 0-54 24.2-54 0 47.8 54 90 54 90s54-42.2 54-90c0-29.8-24.2-54-54-54Zm0 26c15.5...
```


Arquivo: public/icons.svg

Arquivo fonte public/icons.svg.

Total: 25 linhas

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="bluesky-icon" viewBox="0 0 16 17">
    <g clip-path="url(#bluesky-clip)"><path fill="#08060d" d="M7.75 7.735c-.693-1.348-2.58-3.86-4.334-5.097-1.68-1.18...
    <defs><clipPath id="bluesky-clip"><path fill="#fff" d="M.1.85h15.3v15.3H.1z"/></clipPath></defs>
  </symbol>
  <symbol id="discord-icon" viewBox="0 0 20 19">
    <path fill="#08060d" d="M16.224 3.768a14.5 14.5 0 0 0-3.67-1.153c-.158.286-.343.67-.47.976a13.5 13.5 0 0 0-4.067 ...
  </symbol>
  <symbol id="documentation-icon" viewBox="0 0 21 20">
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="m15.5 13...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M17.167 ...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M3 10a2...
  </symbol>
  <symbol id="github-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M9.356 1.85C5.05 1.85 1.57 5.356 1.57 9.694a7.84 7.84 0 0 0 5.324 7.4...
  </symbol>
  <symbol id="social-icon" viewBox="0 0 20 20">
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M12.5 6...
    <path fill="none" stroke="#aa3bfff" stroke-linecap="round" stroke-linejoin="round" stroke-width="1.35" d="M2.5 16...
  </symbol>
  <symbol id="x-icon" viewBox="0 0 19 19">
    <path fill="#08060d" fill-rule="evenodd" d="M1.893 1.98c.052.072 1.245 1.769 2.653 3.771l2.892 4.114c.183.261.333...
  </symbol>
</svg>
```

Arquivo: public/manifest.webmanifest

Arquivo fonte public/manifest.webmanifest.

Total: 18 linhas

```
{
  "name": "SaudeConnect",
  "short_name": "SaudeConnect",
  "description": "Portal integrado para pacientes, unidades de saude e administradores.",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#f5f7fb",
  "theme_color": "#0f8b8d",
  "icons": [
    {
      "src": "/favicon.svg",
      "sizes": "any",
      "type": "image/svg+xml",
      "purpose": "any maskable"
    }
  ]
}
```

Arquivo: src/assets/react.svg

Arquivo fonte src/assets/react.svg.

Total: 1 linhas

```
<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" aria-hidden="true" role="img" clas...
```

Arquivo: src/assets/vite.svg

Arquivo fonte src/assets/vite.svg.

Total: 2 linhas

```
<svg xmlns="http://www.w3.org/2000/svg" width="77" height="47" fill="none" aria-labelledby="vite-logo-title" viewBox=...
```